FATEC POMPEIA

Introdução à Computação em Nuvem · ISW055 · 161_SIST. INTELIGENTES_N

P2

Relatório consolidado

arquitetura e atividades do 2º bimestre

Avaliação individual · 2026.2

Gabriel Castão Graciano

Produto: **CampusSwap**
https://github.com/Gabriel33-fis/campuswap

Prof. Allan Lincoln Rodrigues Siriani
Pompeia, 13/11/2026

Sumário

1 Introdução 2
2 Metodologia 2
3 Arquitetura 2
3.1 Componentes 2
3.2 Onde roda 4
3.3 Fluxo de uma requisição 4
3.4 Fluxo de assinatura 5
3.5 Modelo de dados 5
3.6 Segurança 6
3.7 Observabilidade e CI/CD 8
3.8 Camadas IaaS/PaaS/SaaS 10
4 Quadro de entregas 10
5 Atividades realizadas 11
5.1 Atividade 7 — Plano Premium: cobrança com Stripe 11
5.2 Atividade 8 — SaaS, PaaS, IaaS e a documentação do projeto 13
5.3 Atividade 9 — Fork e plano de negócio 15
5.4 Atividade 10 — Seu negócio no ar: o domínio no lugar dos filmes 17
5.5 Atividade 11 — SaaS completo: planos, limites e assinatura com Stripe 19
6 Considerações finais 25
7 Declaração de autoria 25

1 Introdução

A disciplina de Introdução à Computação em Nuvem (ISW055) trabalha a concepção, a execução e a entrega contínua de aplicações modernas com microsserviços, contêineres Docker e serviços gerenciados. Ao longo do semestre, uma aplicação acadêmica inicial, o Catálogo Tom Hanks, foi transformada em um produto de software como serviço (SaaS) com cobrança recorrente.

O produto é o **CampuSwap**, plataforma para a comunidade acadêmica negociar, trocar e emprestar materiais didáticos (livros, apostilas, calculadoras), reduzindo o custo de estudar e a ociosidade desses itens ao fim de cada semestre. O plano Free permite 3 anúncios por mês; o plano Pro, de R$ 9,90 por mês, remove o limite.

Este relatório descreve a arquitetura da versão que está no ar em https://gabriel-graciano-isw055.lapps.studio, a mesma que foi usada para coletar as evidências, e documenta, atividade por atividade (7 a 11), o que foi entregue, quando, com que evidência e quais dificuldades surgiram, inclusive as entregas feitas com atraso.

2 Metodologia

O desenvolvimento foi feito em ambiente local (Zorin OS) com Docker Compose, e a publicação em uma máquina virtual da disciplina, por meio de uma stack do Portainer criada a partir do repositório Git (arquivo `docker-compose.prod.yml`). As atividades 7 e 8 foram feitas no repositório original `catalogo-tom-hanks`; a partir da Atividade 9 o histórico foi espelhado (`git clone --bare` e `git push --mirror`) para o repositório `campuswap`, preservando os hashes dos commits. As imagens dos três serviços são publicadas no GitHub Container Registry pelo GitHub Actions.

As entregas formais foram registradas por issues que mencionam o professor (`@siriani`), associadas a commits. **Origem das datas e horas:** o GitHub (issues, commits, `.patch` e dicas de data ao passar o mouse) mostra horário de Brasília (GMT-3); o painel do Stripe, o painel admin do CampuSwap e a tabela `eventos_stripe` mostram UTC (3 horas à frente de Brasília); os comandos de terminal mostram a hora do sistema. Quando o GitHub exibe só "há X dias", a data exata vem da página `.patch` do commit (campo `Date`). Quando houve diferença entre o dia previsto e o dia efetivo, a situação é informada como atraso, com o motivo.

Para diagnosticar falhas usei `docker logs`, `curl`, consultas diretas ao MariaDB e o painel de entregas de webhooks do Stripe. Para cada atividade, o horário de uma evidência é o que aparece no próprio print; nenhuma data foi alterada.

3 Arquitetura

3.1 Componentes

O CampuSwap é composto por três microsserviços FastAPI e três serviços de apoio, todos em contêineres Docker. Somente o `catalogo_service` é exposto ao público (porta 8207 do host, 8000 no contêiner); ele entrega o front-end, a API de anúncios e atua como proxy das rotas de autenticação, checkout e webhook para o `auth_service`.

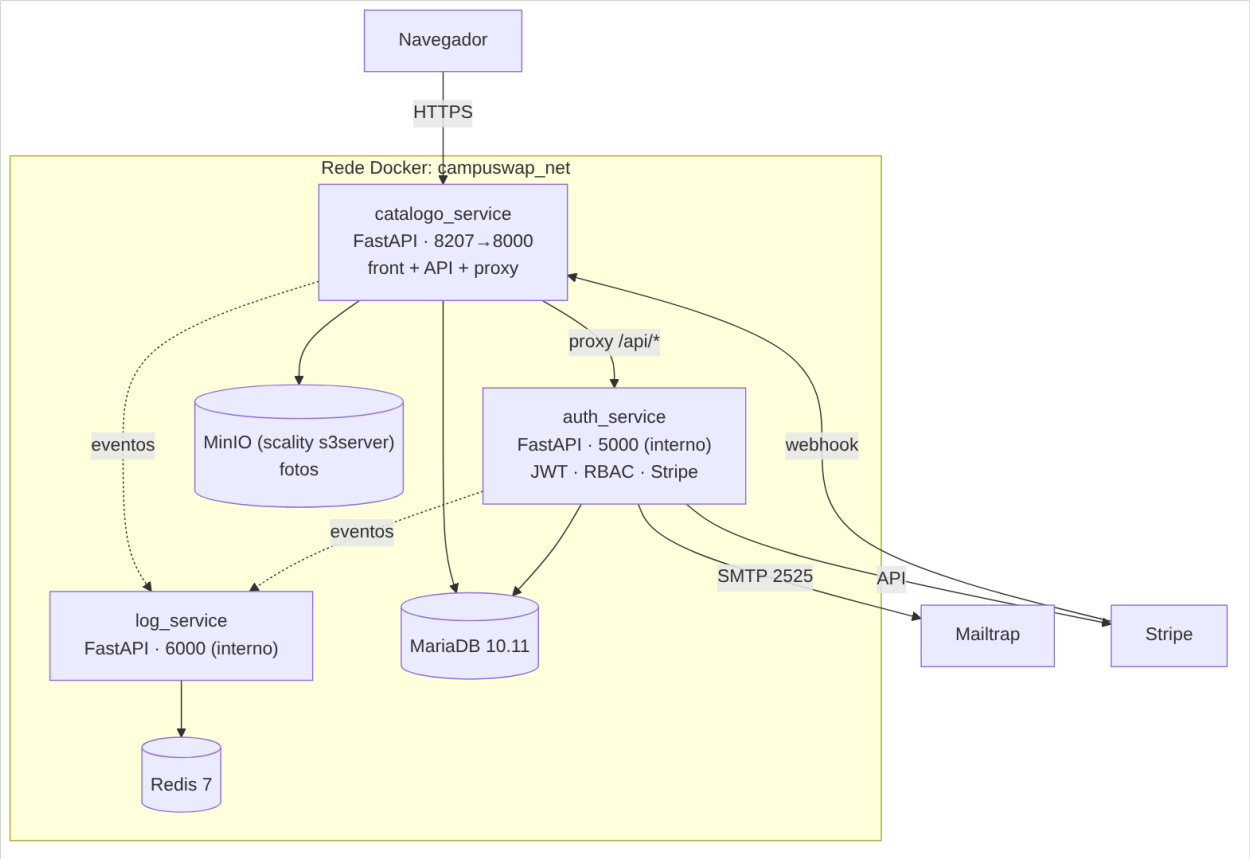


Figura 1: Diagrama de componentes do CampuSwap em produção

Componente	Responsabilidade	Tecnologia	Porta
catalogo_service	Front-end HTML/JS, API de anúncios e fotos, proxy para o `auth_service`, painel admin	FastAPI, Python	8207 (público)
auth_service	Cadastro, login JWT, RBAC, limite do plano Free, checkout, portal e webhook do Stripe, recuperação de senha	FastAPI, SQLAlchemy	5000 (interna)
log_service	Recebe eventos de auditoria	FastAPI, Redis	6000 (interna)
Redis	Fila de auditoria e cache da listagem	Redis 7	interna
MariaDB	Usuários, anúncios, tokens de recuperação, eventos do Stripe	MariaDB 10.11	interna
MinIO	Armazenamento S3 das fotos (bucket `campuswap-fotos`)	scality/s3server	interna
Mailtrap	SMTP de testes para e-mails de recuperação de senha	SaaS externo	saída 2525
Stripe	Checkout, assinaturas, portal e webhooks (modo de teste)	SaaS externo	webhook para o catálogo

3.2 Onde roda

A aplicação roda em uma máquina virtual (IaaS) da disciplina, na stack `campuswap` do Portainer (`portainer.lapps.studio`), publicada em https://gabriel-graciano-isw055.lapps.studio. A stack é criada a partir do repositório Git e do arquivo `docker-compose.prod.yml`, na rede `campuswap_net`, com limites de memória por contêiner (MariaDB 384 MB; catálogo, autenticação e MinIO 256 MB; Redis e log 128 MB). Os segredos (`JWT_SECRET`, `STRIPE_SECRET_KEY`, `STRIPE_WEBHOOK_SECRET`, preço do plano, senhas do banco e do SMTP) são variáveis de ambiente da stack no Portainer; o repositório guarda apenas os nomes das variáveis.

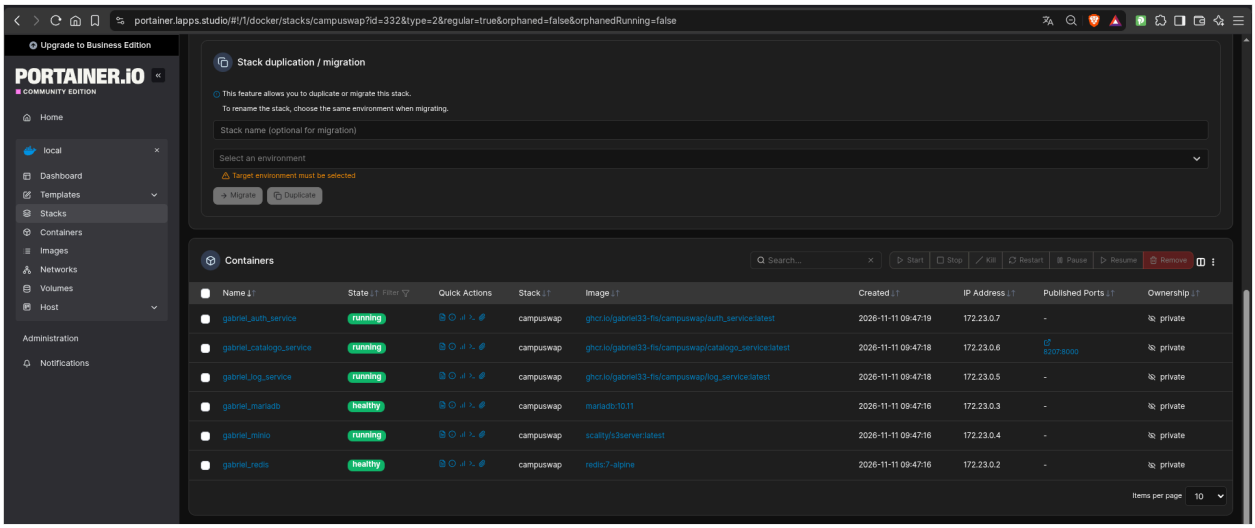


Figura 2: Stack `campuswap` no Portainer: seis contêineres em execução

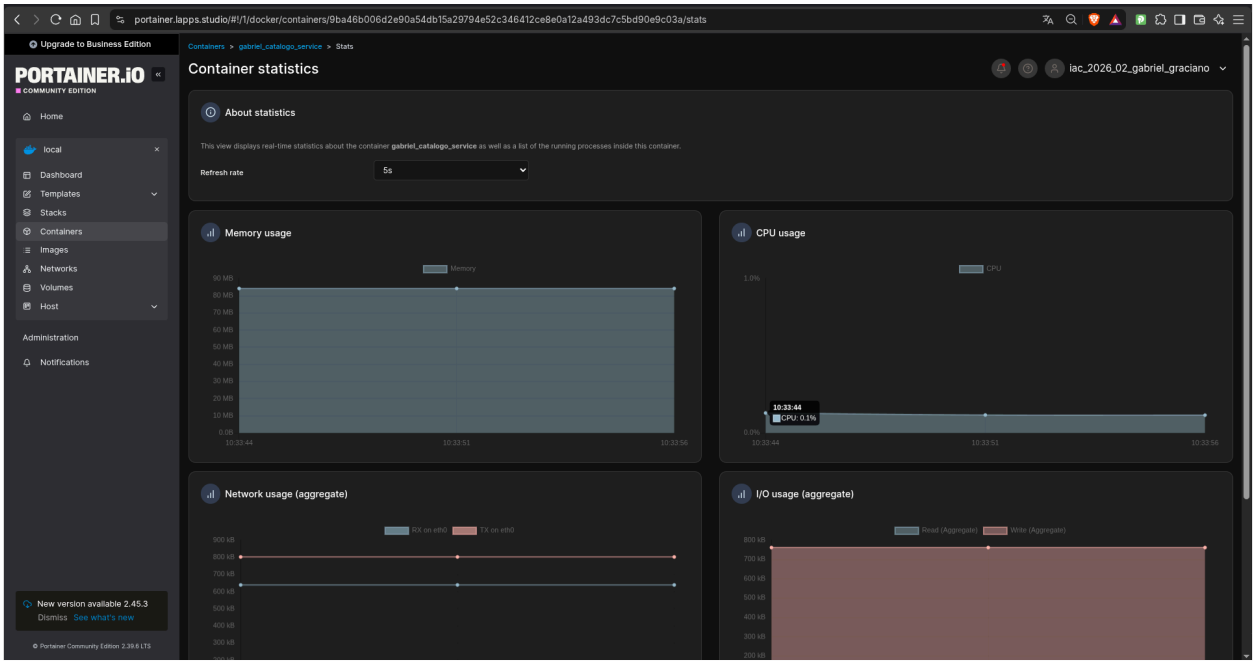


Figura 3: Uso de memória do contêiner `gabriel_catalogo_service` no Portainer

3.3 Fluxo de uma requisição

1. O navegador acessa https://gabriel-graciano-isw055.lapps.studio e o `catalogo_service` devolve o `index.html`.
2. O JavaScript chama a API (por exemplo `GET /api/itens`) no mesmo domínio.

3. Em rotas protegidas, o serviço lê `Authorization: Bearer <token>`, valida o JWT e aplica o papel (RBAC). Rotas de `login`, `register`, recuperação de senha, checkout e portal são encaminhadas ao `auth_service` pela rede interna.
4. O catálogo consulta o MariaDB (SQLAlchemy), com cache da listagem no Redis quando disponível, e as fotos vêm do MinIO.
5. Eventos relevantes (login, publicação, acesso negado, eventos do Stripe) são enviados ao `log_service`, que os guarda no Redis e os expõe ao painel admin.
6. O catálogo responde o JSON ao navegador.

3.4 Fluxo de assinatura

1. O aluno clica em assinar o plano Pro; o front chama `POST /api/checkout`, que o catálogo encaminha ao `auth_service`. Este cria o cliente no Stripe, se necessário, e uma Checkout Session em modo `subscription` com o `price_...` configurado.
2. O aluno paga no ambiente do Stripe com o cartão de teste 4242 4242 4242 4242.
3. O Stripe envia `POST /api/assinaturas/webhook` ao catálogo, que repassa o corpo e o cabeçalho `Stripe-Signature` ao `auth_service`.
4. O `auth_service` valida a assinatura com `STRIPE_WEBHOOK_SECRET` e registra o `event_id` na tabela `eventos_stripe`, o que evita processar o mesmo evento duas vezes.
5. Em `checkout.session.completed`, o usuário passa a `plano = pro`, `status_assinatura = active` e `is_premium = true`. Eventos de atualização, cancelamento e falha de pagamento mantêm o status em sincronia.
6. O portal do cliente (`/api/assinaturas/portal`) permite gerenciar ou cancelar.

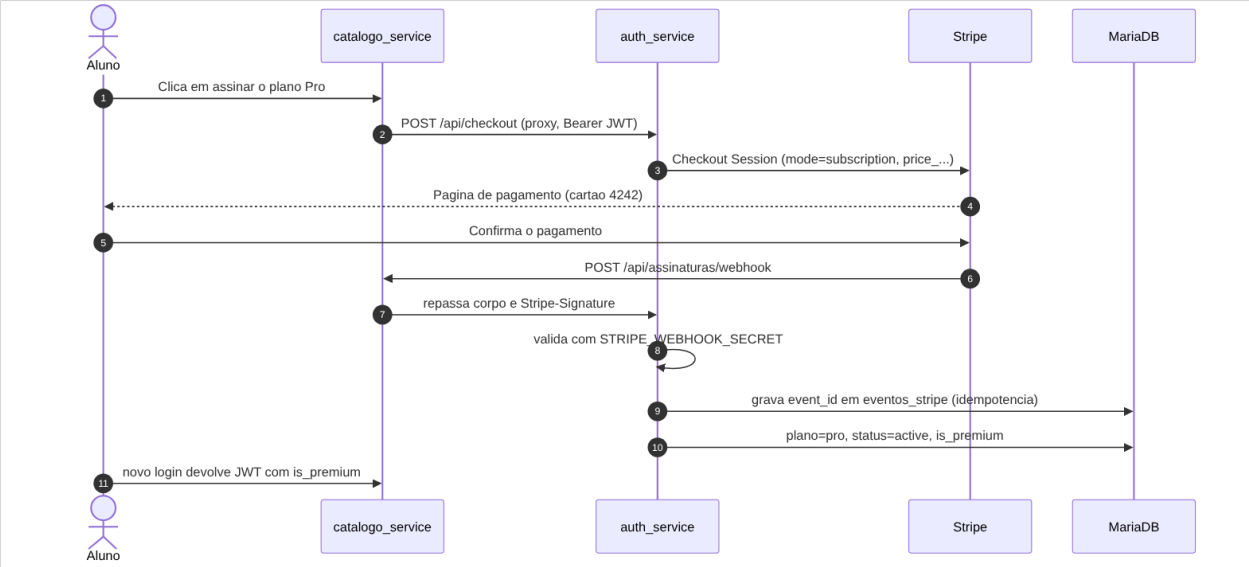


Figura 4: Fluxo de assinatura com webhook do Stripe

3.5 Modelo de dados

- `usuarios`: `id`, `nome`, `email`, `senha_hash` (bcrypt), `papel` (aluno, moderador, admin), `plano` (free ou pro), `status_assinatura`, `is_premium`, `stripe_customer_id`, `stripe_subscription_id`.
- `itens` (anúncios): `id`, `usuario_id`, `titulo`, `descricao`, `categoria_id`, `condicao`, `modalidade`, `preco`, `foto_key`, `imagem_url`, `status`, `criado_em`.
- `recuperacoes_senha`: `usuario_id`, `token` único, `expira_em`, `usado`.
- `eventos_stripe`: `event_id` único, `tipo`, `processado_em` (idempotência dos webhooks).
- A trilha de auditoria fica no Redis, gerida pelo `log_service`.

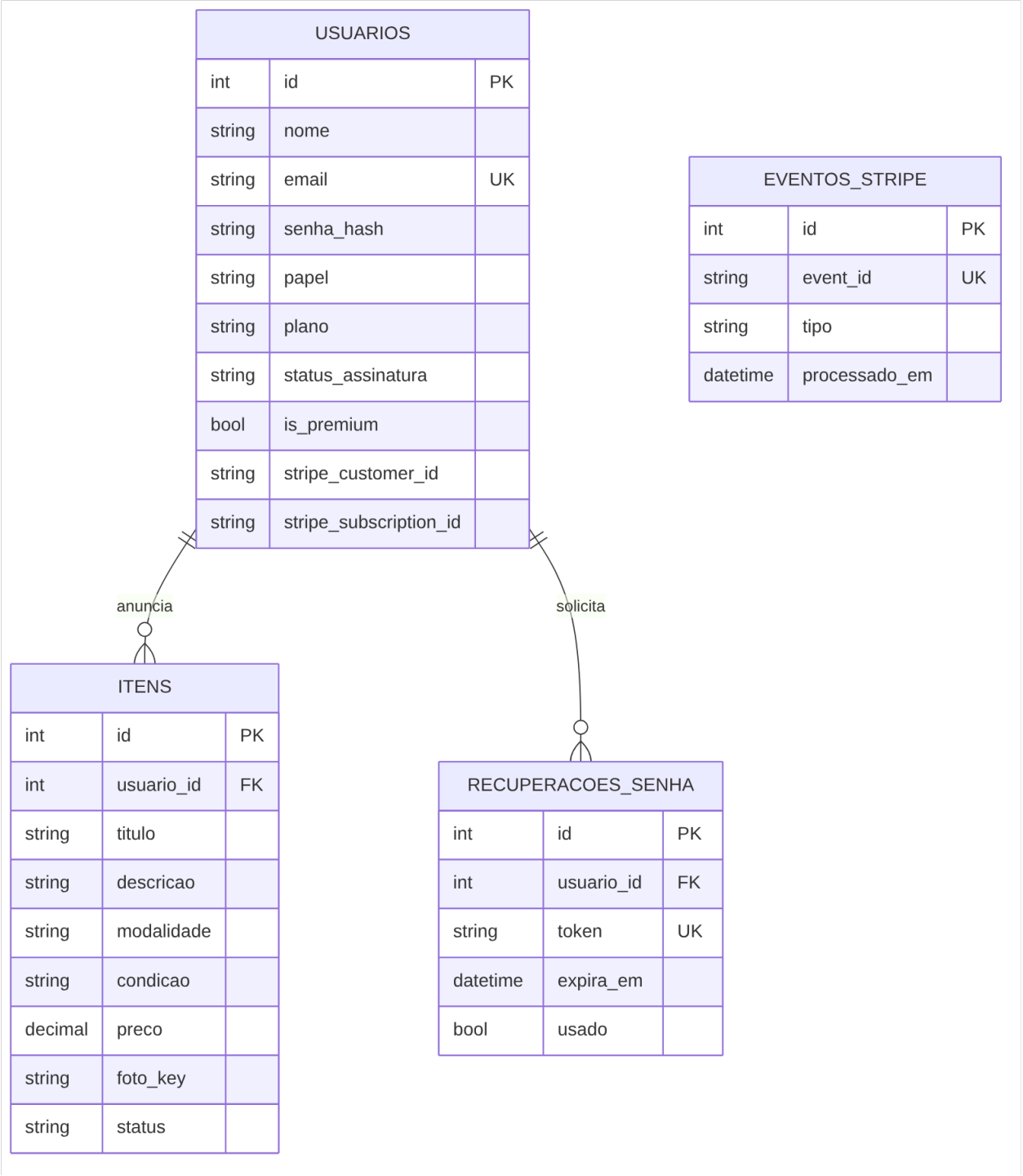


Figura 5: Modelo entidade-relacionamento (diagrama a partir de `models.py`; as ligações por `usuario_id` são lógicas, sem chave estrangeira física)

3.6 Segurança

Ação	Admin	Aluno Free	Aluno Pro
Ver anúncios públicos	Sim	Sim	Sim
Publicar anúncios	Sim	Até 3 por mês	Sem limite
Editar ou excluir os próprios anúncios	Sim	Sim	Sim

Ação	Admin	Aluno Free	Aluno Pro
Excluir anúncios de outros alunos	Sim	Não	Não
Painel admin e logs de auditoria	Sim	Não (403)	Não (403)

- **Papéis:** o JWT carrega `usuario_id`, `papel` e `is_premium`. Rotas administrativas exigem `admin` e respondem 401 (sem token válido) ou 403 (papel insuficiente). O cadastro público sempre cria `aluno`: o campo `papel` enviado pelo cliente é ignorado (correção da PR #13).
- **Posse do anúncio:** a exclusão compara o `usuario_id` do token com o dono do registro.
- **Webhook:** só é aceito com assinatura válida do Stripe; eventos repetidos são ignorados.
- **Segredos:** ficam em variáveis do Portainer e no `.env` local, que está no `.gitignore`. Chaves que apareceram em prints ou conversas durante o desenvolvimento são de modo de teste e devem ser rotacionadas após a entrega.
- **Rede:** `auth_service`, `log_service`, banco, Redis e MinIO não publicam portas no host.

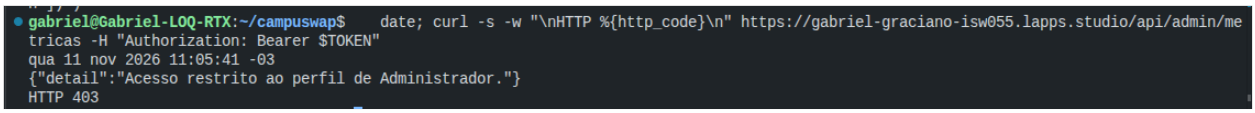


Figura 6: Chamada à rota admin com perfil de aluno em produção, com data e hora do terminal: HTTP 403

Verificação do cadastro em produção. Em 11/11/2026, um teste de cadastro com `"papel":"admin"` feito antes de atualizar a stack ainda criou uma conta com privilégio de administrador, o que confirmou a falha corrigida na PR #13. Depois de recriar os contêineres com a imagem nova, o console do contêiner `gabriel_auth_service` no Portainer mostra a linha do fix (11:56), e uma conta cadastrada com `"papel":"admin"`, em janela anônima, recebeu o aviso “Acesso restrito ao perfil de Administrador” e nenhum dado do painel (11:58). O menu “Painel admin” ainda aparece no front-end para qualquer usuário, mas a API bloqueia os dados.

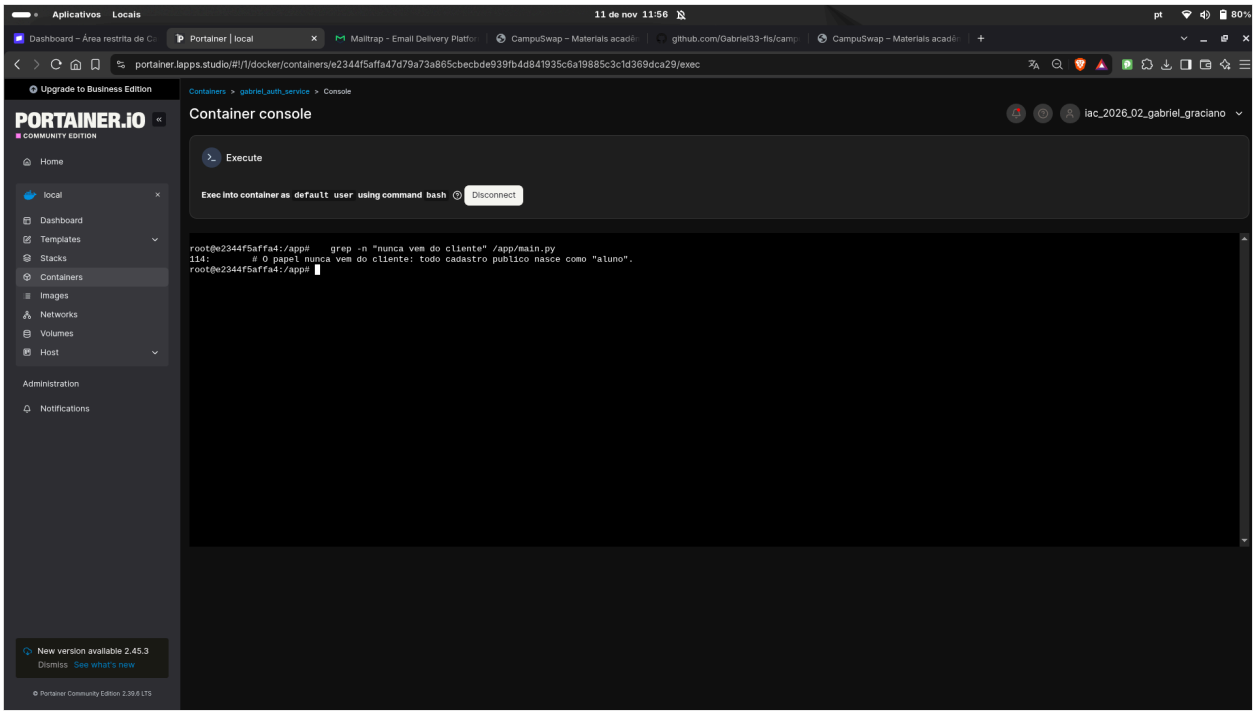


Figura 7: Console do contêiner `gabriel_auth_service` em produção (Portainer, 11:56): o fix está na imagem em execução

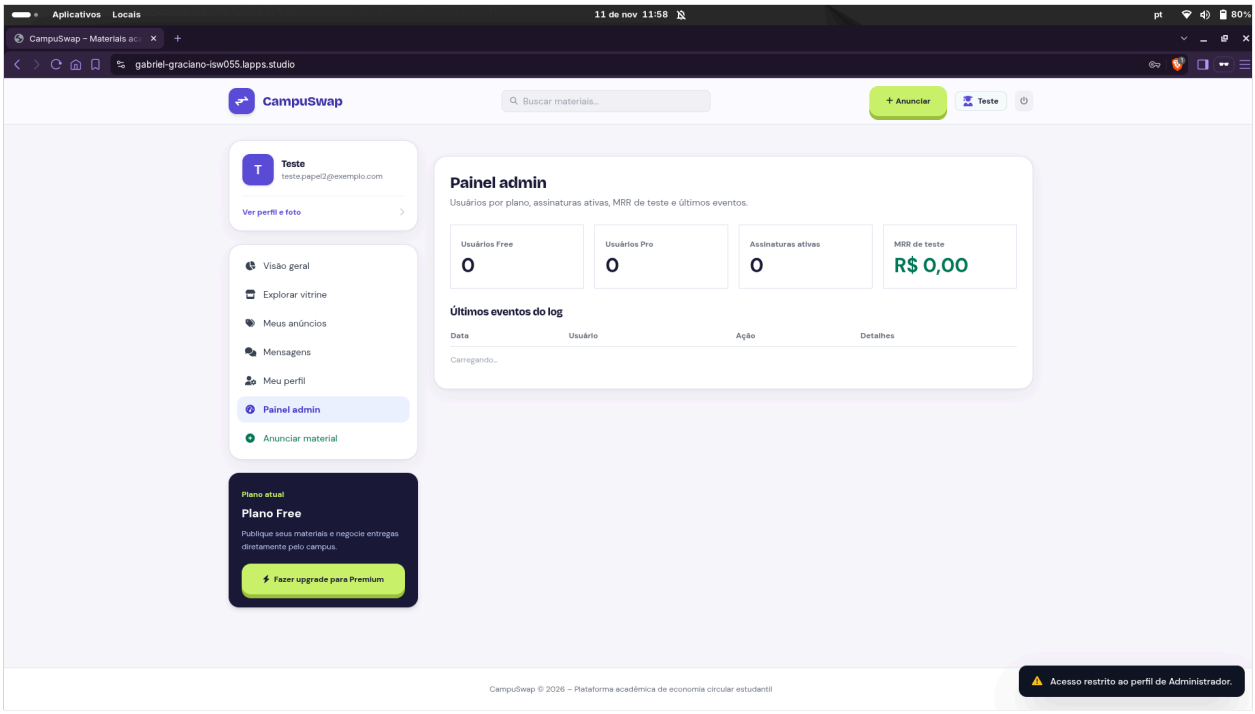


Figura 8: Conta cadastrada com `"papel":"admin"` acessando o painel admin em produção (11:58): acesso negado e sem dados

3.7 Observabilidade e CI/CD

Os três serviços expõem `GET /health` (200 quando as dependências respondem, 503 caso contrário) e `/metrics` (Prometheus), e a documentação OpenAPI em `/docs`. O painel admin lista os últimos eventos de auditoria.

O fluxo `.github/workflows/deploy.yml` roda a cada push na `main`: o job de CI instala as dependências dos três serviços e compila o código (`python -m compileall`); o de CD constrói e publica as imagens `auth_service`, `catalogo_service` e `log_service` no GHCR com as tags `latest` e `sha-<commit>`. A stack do Portainer usa essas imagens. O CI anterior executava testes herdados do Catálogo Tom Hanks, que falhavam no novo repositório e bloqueavam a publicação; por isso foi substituído pela verificação de compilação.

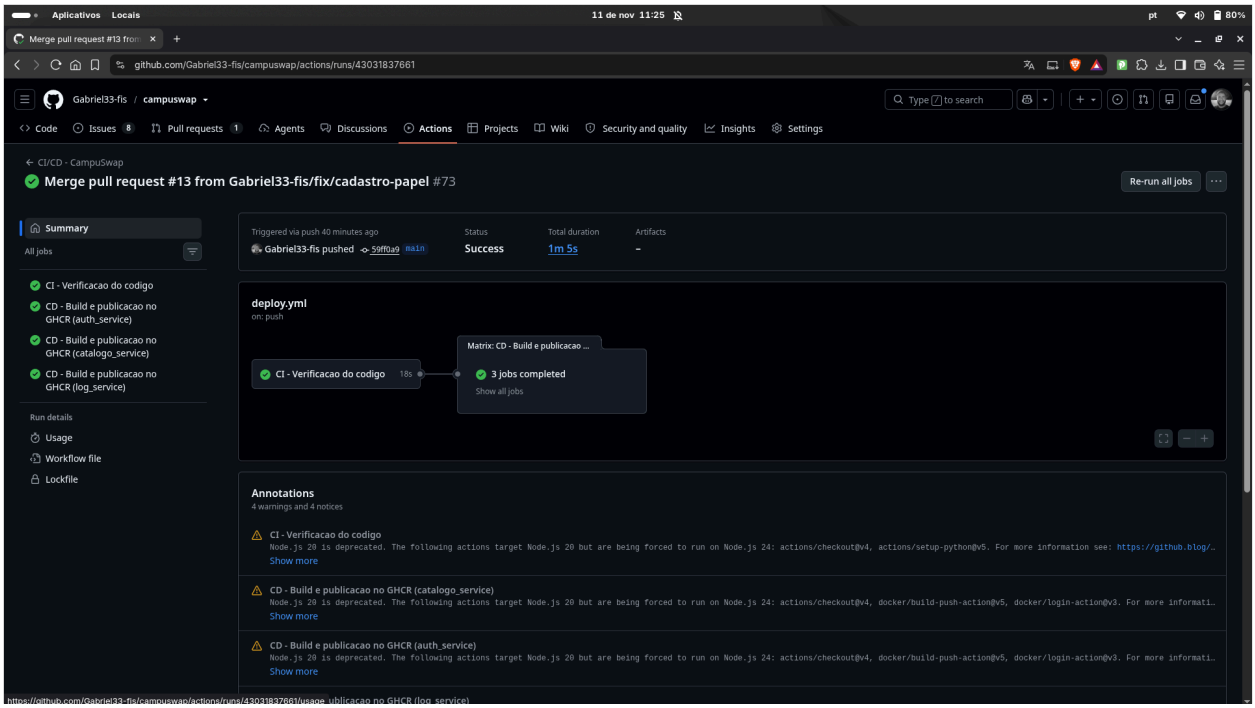


Figura 9: GitHub Actions: execução do commit 59ff0a9 (merge da PR #13) com CI e CD verdes, em 11/11/2026 (relógio do sistema visível)

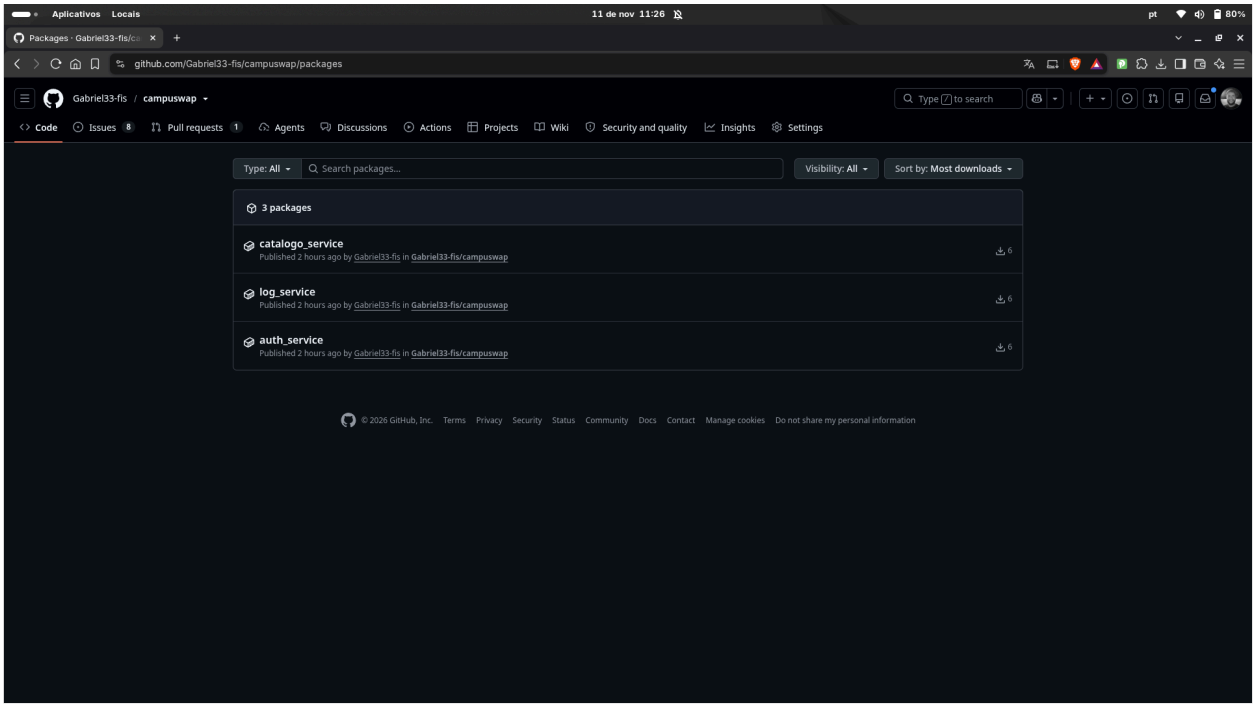


Figura 10: Pacotes das três imagens publicados no GHCR pelo CD, em 11/11/2026

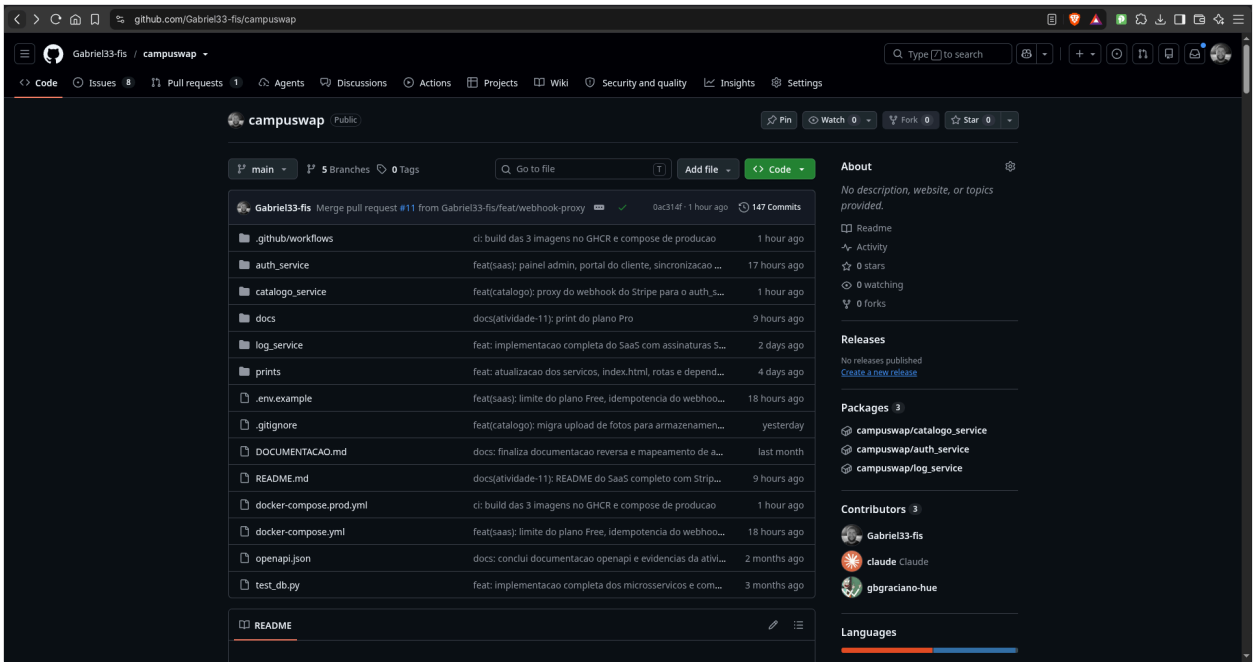


Figura 11: Página inicial do repositório `campuswap`, com os pacotes publicados no GHCR e a PR #11

3.8 Camadas IaaS/PaaS/SaaS

Camada	No projeto	Quem gerencia
IaaS	Máquina virtual Linux, rede, CPU, memória e disco.	Fatec / professor
PaaS	Docker Engine e Portainer rodando a stack, GHCR e GitHub Actions para build e imagens, MariaDB, Redis e MinIO como serviços da stack.	Aluno (stack e configuração)
SaaS	O próprio CampuSwap entregue aos alunos; e os SaaS consumidos: Stripe, Mailtrap e GitHub.	Aluno (produto) e fornecedores

4 Quadro de entregas

Nº	Atividade	Data planejada	Data realizada	Situação
7	Plano Premium: cobrança com Stripe	09/10/2026	11/10/2026 16:44 (issue)	entregue com atraso (2 dias)
8	SaaS, PaaS, IaaS e a documentação do projeto	16/10/2026	16/10/2026 19:48 (issue)	entregue no prazo
9	Fork e plano de negócio	23/10/2026	23/10/2026 19:38	entregue no prazo
10	Seu negócio no ar: o domínio no lugar dos filmes	06/11/2026	06/11/2026 22:25	entregue no prazo
11	SaaS completo: planos, limites e assinatura com Stripe	13/11/2026	11/11/2026 00:44	entregue no prazo

5 Atividades realizadas

5.1 Atividade 7 — Plano Premium: cobrança com Stripe

Descrição	Plano pago em modo de teste: Checkout Session do Stripe e webhook marcando o usuário como premium.
Data planejada	09/10/2026
Data realizada	11/10/2026 16:44 (issue)
Situação	entregue com atraso (2 dias)
Evidência	Issue \#10 do catalogo-tom-hanks (11/10/2026 16:44), commit 7ecdea2 (10/10/2026 20:19) e prints do checkout e do benefício ativo
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/issues/10

Motivo do atraso: por falta de tempo não consegui entregar no dia. O código foi commitado em 10/10 (1 dia após o prazo) e a issue de entrega só foi aberta em 11/10 às 16:44 (2 dias após o prazo); considero a data da issue como a da entrega.

O que foi feito. Integração com o Stripe em sandbox no projeto inicial: rota de checkout no backend, redirecionamento ao pagamento e webhook para `checkout.session.completed` que muda o usuário de `free` para premium. No catálogo da época, a conta gratuita era limitada em favoritos e a assinatura liberava o uso ilimitado com selo Premium.

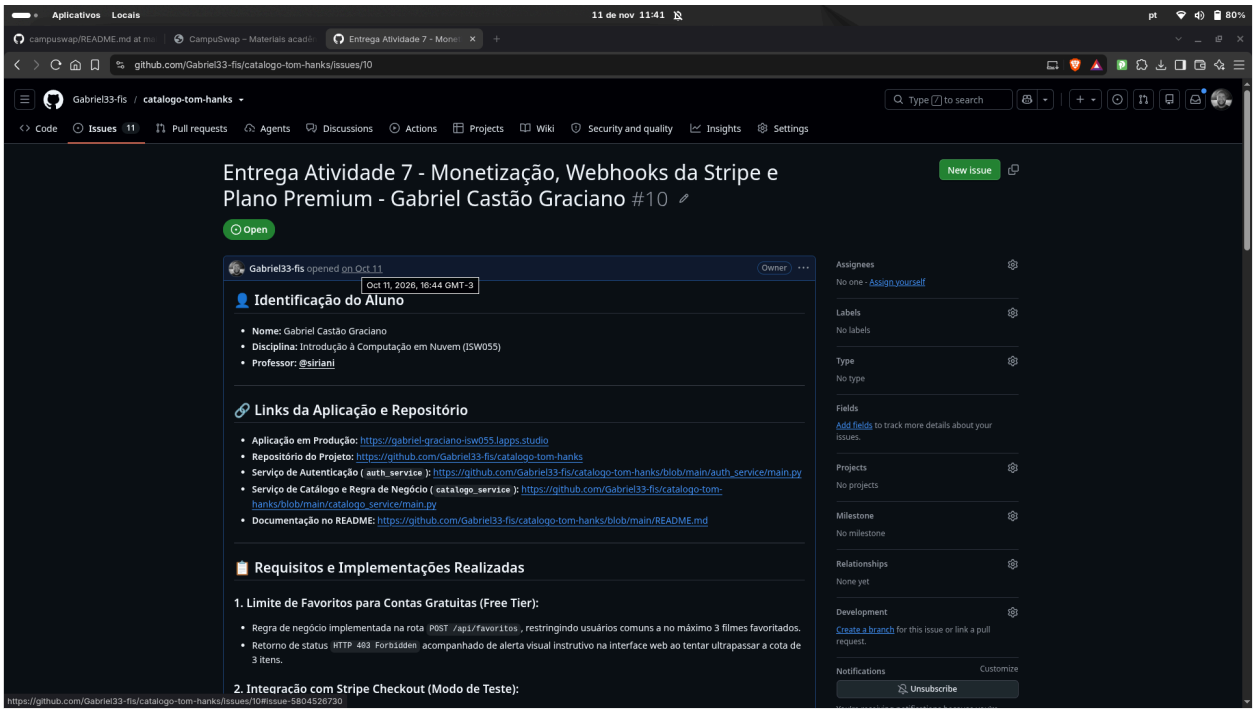


Figura 12: Atividade 7: issue #10 de entrega, com a data e hora ao passar o mouse (11 de outubro de 2026, 16:44 GMT-3)

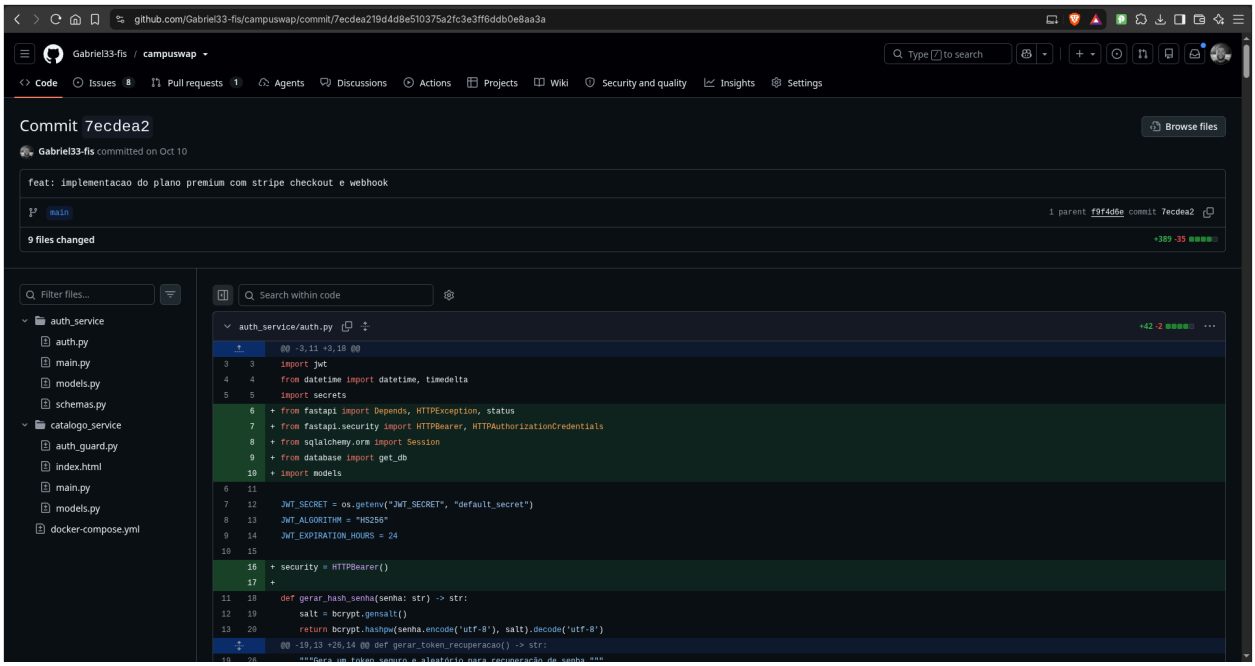


Figura 13: Atividade 7: commit 7ecdea2 (implementação do plano premium com Stripe)

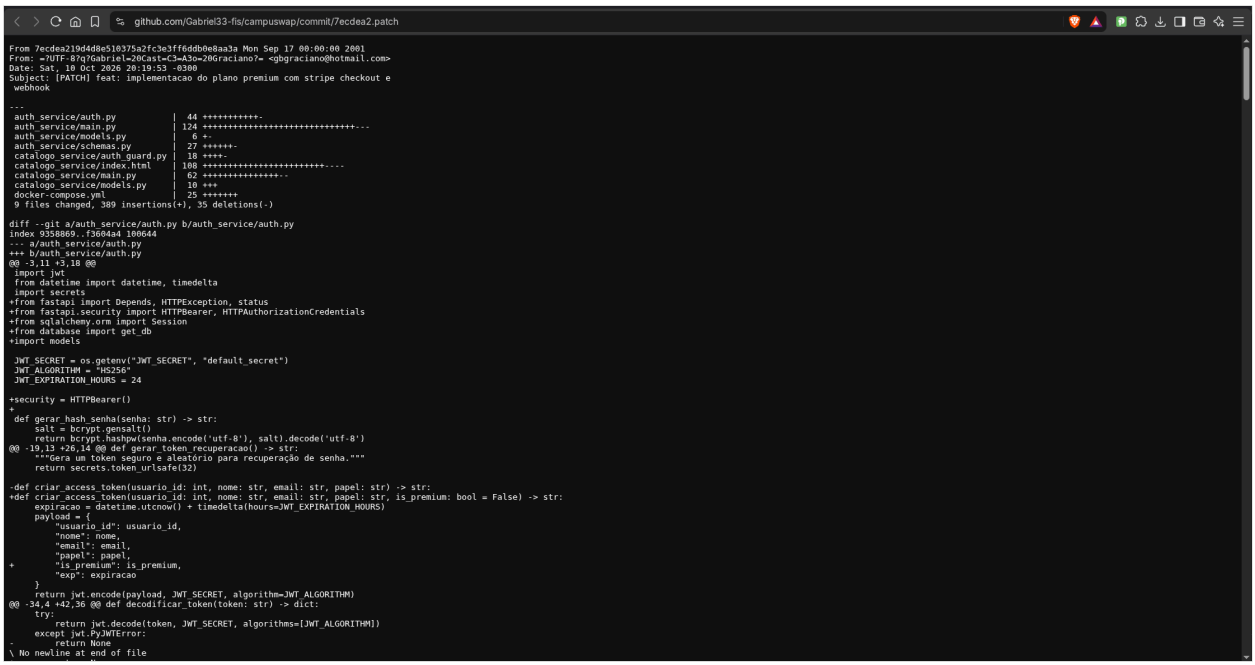
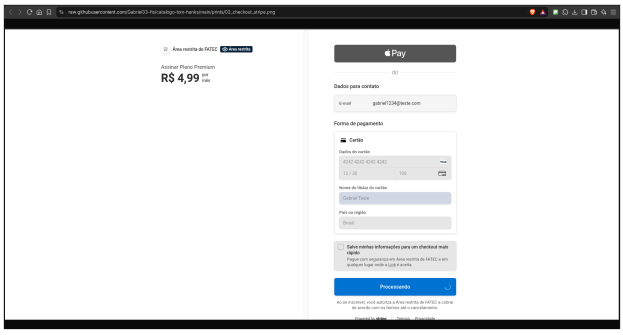


Figura 14: Atividade 7: `.patch` do commit 7ecdea2, com `Date: Sat, 10 Oct 2026 20:19:53 -0300`



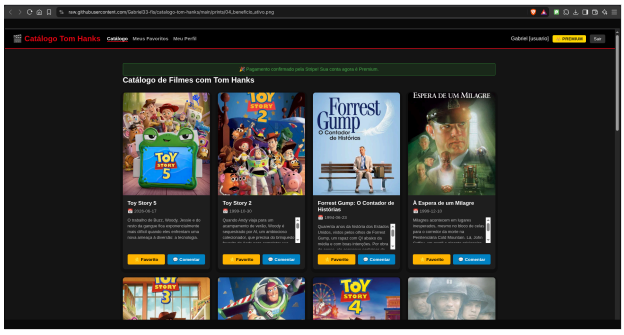


Figura 15: Atividade 7: checkout do Stripe em modo de teste e selo Premium ativo

Dificuldades. Entregar o webhook ao contêiner exigiu configurar o segredo de assinatura (`endpoint_secret`). Também foi preciso incluir `is_premium` no token do login e ajustar as URLs de retorno do Stripe.

5.2 Atividade 8 — SaaS, PaaS, IaaS e a documentação do projeto

Descrição	Camadas de nuvem do projeto e documentação reversa (especificação, arquitetura, regras de negócio, modelo de dados).
Data planejada	16/10/2026
Data realizada	16/10/2026 19:48 (issue)
Situação	entregue no prazo
Evidência	Issue \#11 do catalogo-tom-hanks (16/10/2026 19:48), commit 63eae93 (19:46) e README com o link da documentação
Link	https://github.com/Gabriel33-fis/catalogo-tom-hanks/issues/11

O que foi feito. Documentação reversa do sistema (`DOCUMENTACAO.md`): especificação funcional, arquitetura, regras de negócio e modelo de dados, com a classificação dos recursos em IaaS, PaaS e SaaS. O README do repositório aponta para o documento.

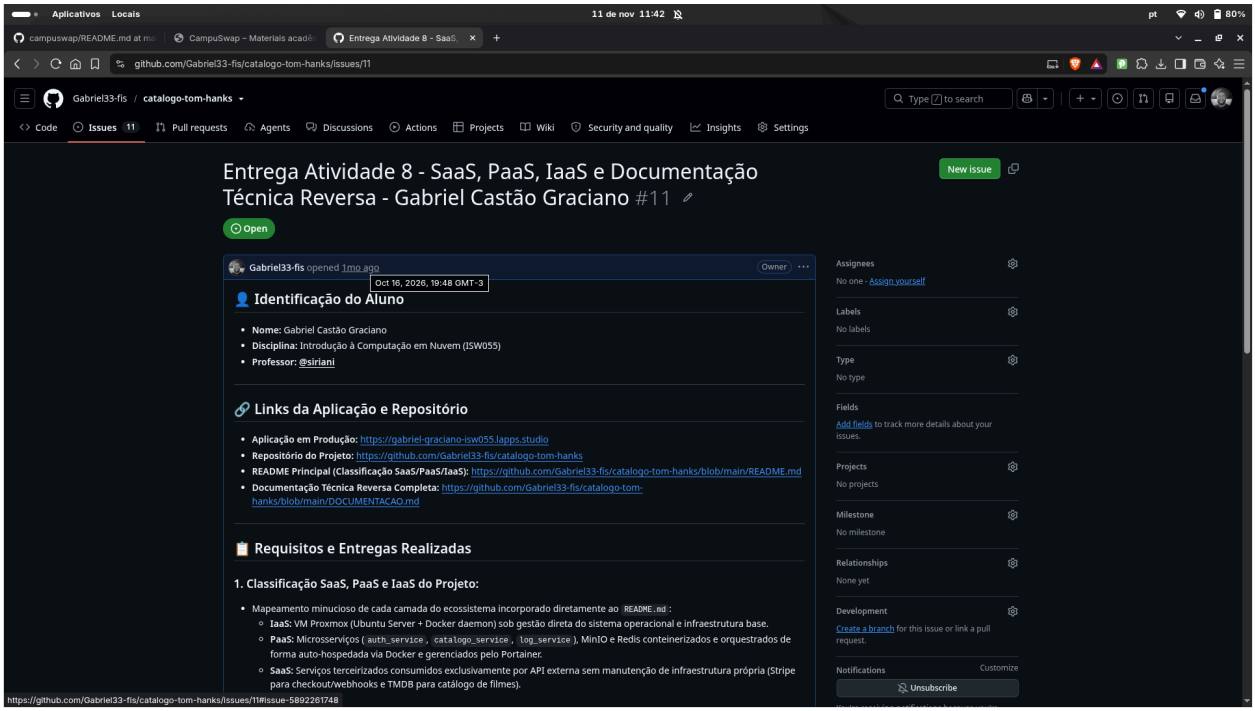


Figura 16: Atividade 8: issue #11 de entrega, com a data e hora ao passar o mouse (16 de outubro de 2026, 19:48 GMT-3)

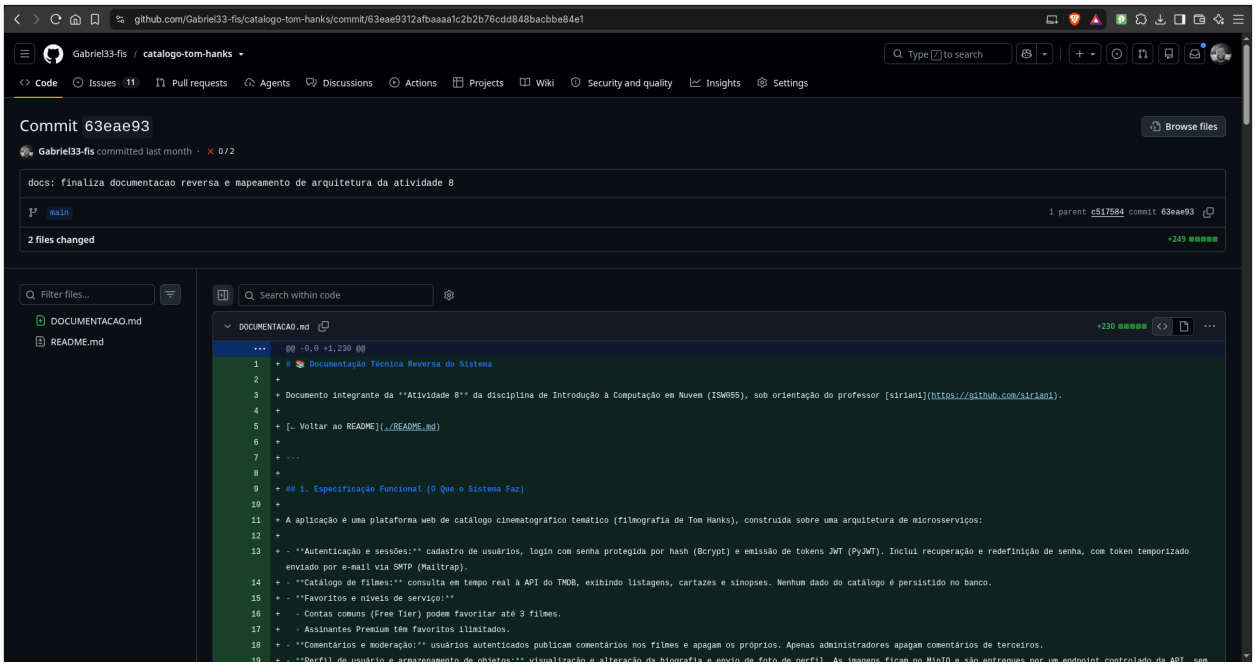


Figura 17: Atividade 8: commit 63eae93 (documentação reversa)



Figura 18: Atividade 8: `.patch` do commit 63eae93, com `Date: Fri, 16 Oct 2026 19:46:30 -0300`

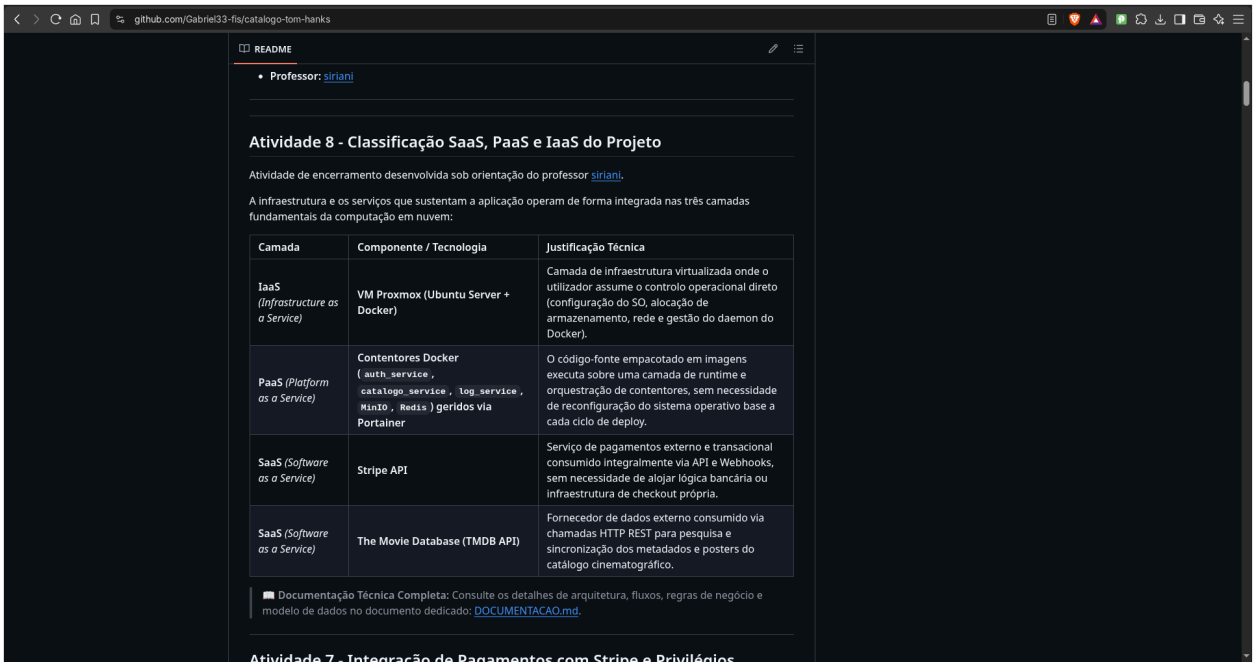


Figura 19: Atividade 8: README do repositório com a tabela de camadas e o link para `DOCUMENTACAO.md`

Dificuldades. Separar onde termina o PaaS (contêineres e imagens) e onde começa o SaaS do projeto exigiu rever os limites de responsabilidade em cada camada.

5.3 Atividade 9 — Fork e plano de negócio

Descrição	Repositório novo com o histórico do anterior, README do produto (dor, persona, Lean Canvas, receita) e backlog em issues.
Data planejada	23/10/2026
Data realizada	23/10/2026 19:38
Situação	entregue no prazo
Evidência	Issue \#6 do repositório campuswap (horário visível), lista de issues e histórico de commits
Link	https://github.com/Gabriel33-fis/campuswap/issues/6

O que foi feito. O repositório `campuswap` foi criado por espelhamento do catálogo, mantendo o histórico. Foram escritos o plano de negócio e o README do produto e abertas as issues do backlog (1 a 5), além da issue de entrega, que menciona o professor.

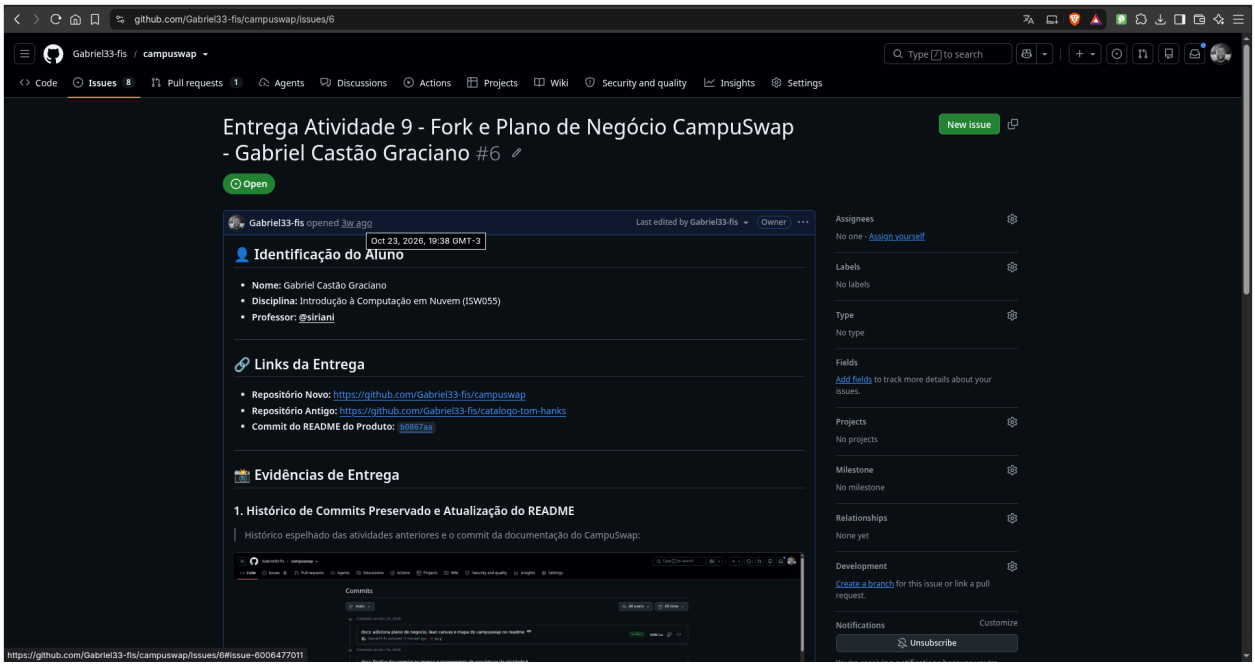
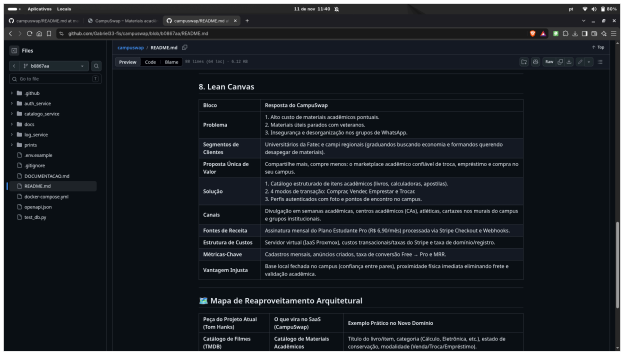


Figura 20: Atividade 9: issue #6 “Entrega Atividade 9” com data e hora (23/10/2026 19:38)



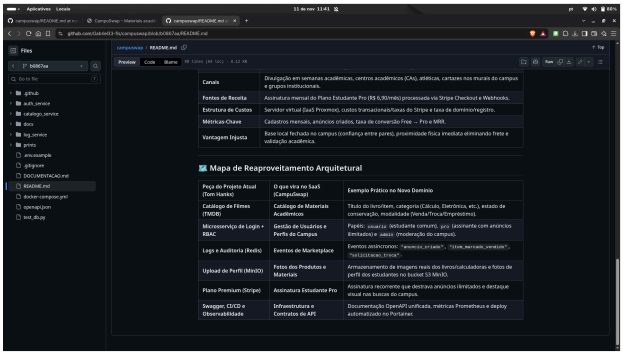


Figura 21: Atividade 9: README do produto no commit b0867aa (Lean Canvas e mapa de reaproveitamento), visto em 11/11/2026

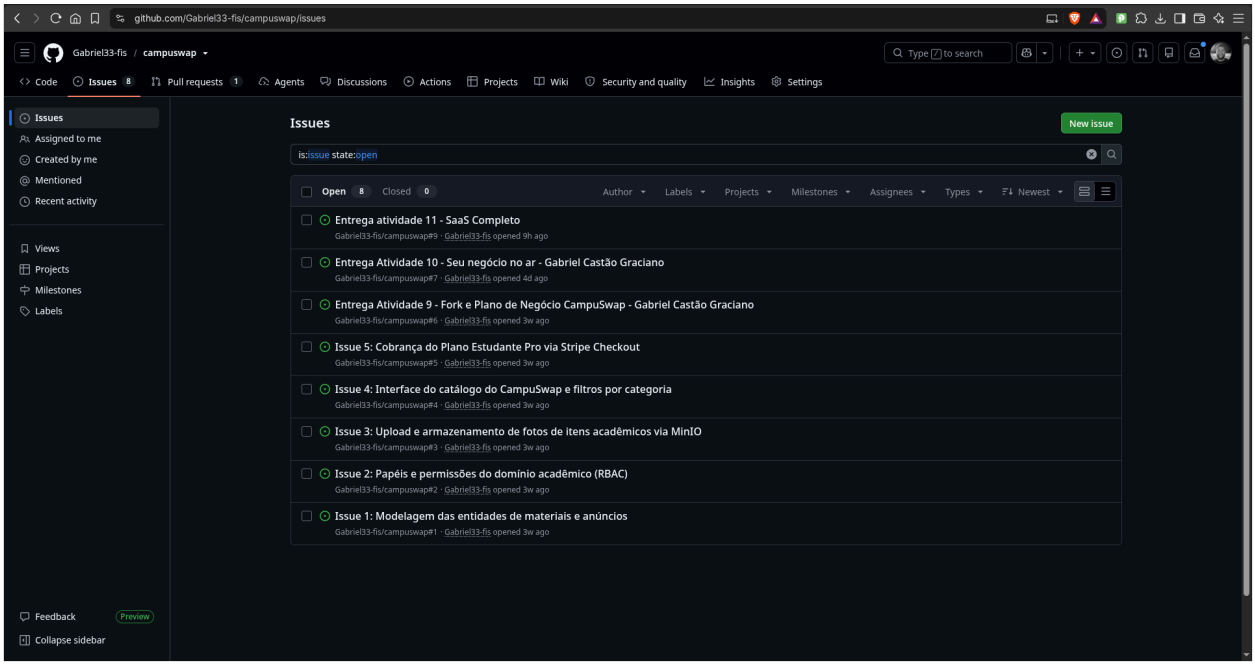


Figura 22: Atividade 9: lista de issues do `campuswap`

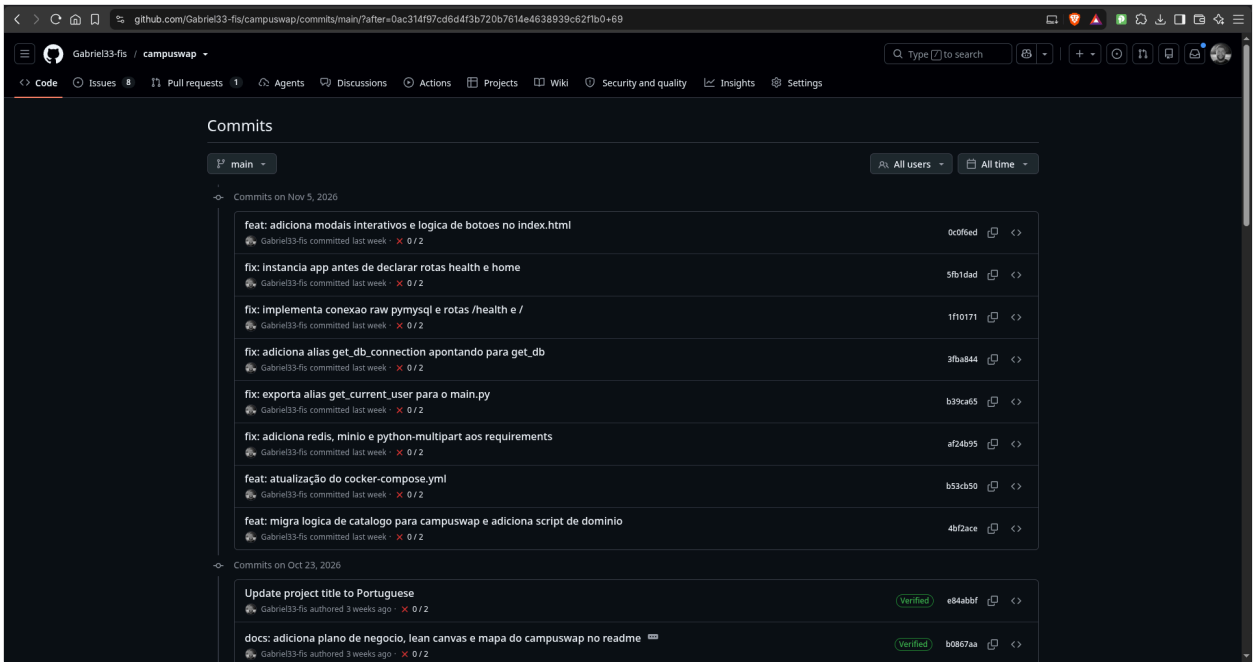


Figura 23: Atividade 9: histórico de commits preservado no repositório novo

Dificuldades. Migrar todas as referências (branches e tags) sem perder a autoria exigiu usar o espelhamento completo do Git em vez de um simples clone.

5.4 Atividade 10 — Seu negócio no ar: o domínio no lugar dos filmes

Descrição	Stack apontando para o repositório novo, tabelas e API do domínio, papéis, isolamento por usuário, eventos no log, upload e landing page.
Data planejada	06/11/2026
Data realizada	06/11/2026 22:25
Situação	entregue no prazo
Evidência	Issue \#7 (06/11/2026 22:25), commit c6e7b5f (22:18) e prints da landing, do 403 e dos eventos
Link	https://github.com/Gabriel33-fis/campuswap/issues/7

O que foi feito. O modelo de filmes foi substituído pelo domínio do CampuSwap: tabelas de usuários e itens, API REST de anúncios, upload de fotos, landing page, papéis `aluno`, `moderador` e `admin`, bloqueio de exclusão de anúncios alheios (403) e eventos de auditoria no `log_service`.

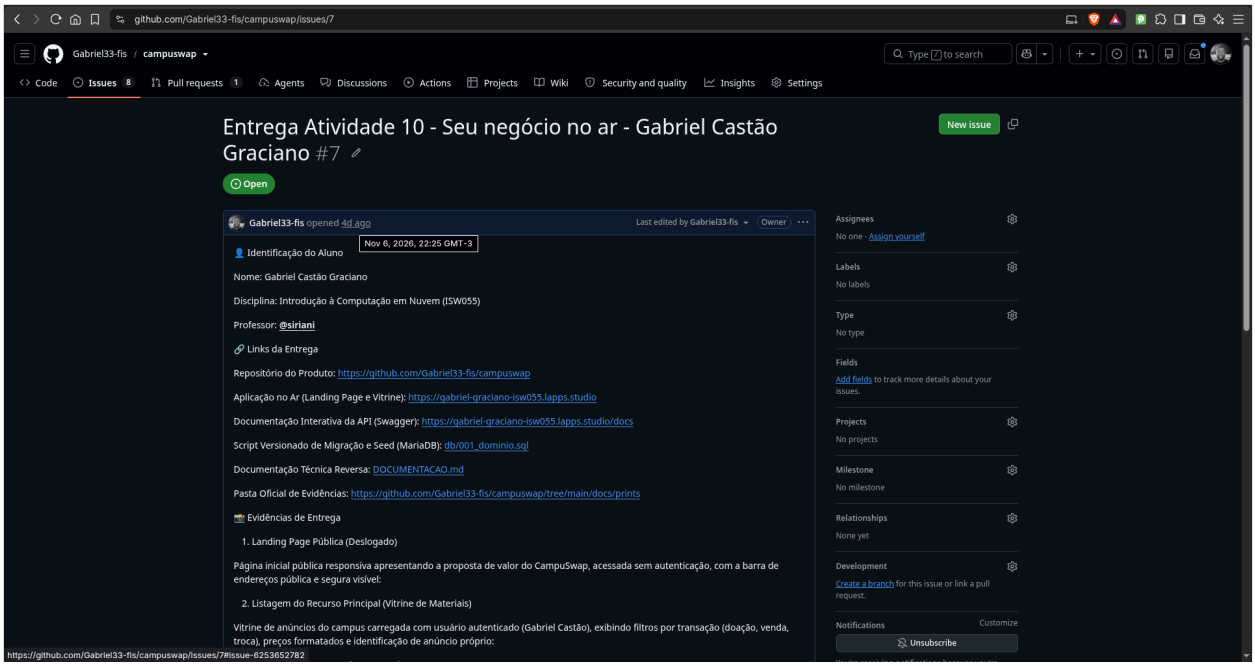


Figura 24: Atividade 10: issue #7 de entrega, com a data e hora ao passar o mouse (6 de novembro de 2026, 22:25 GMT-3)

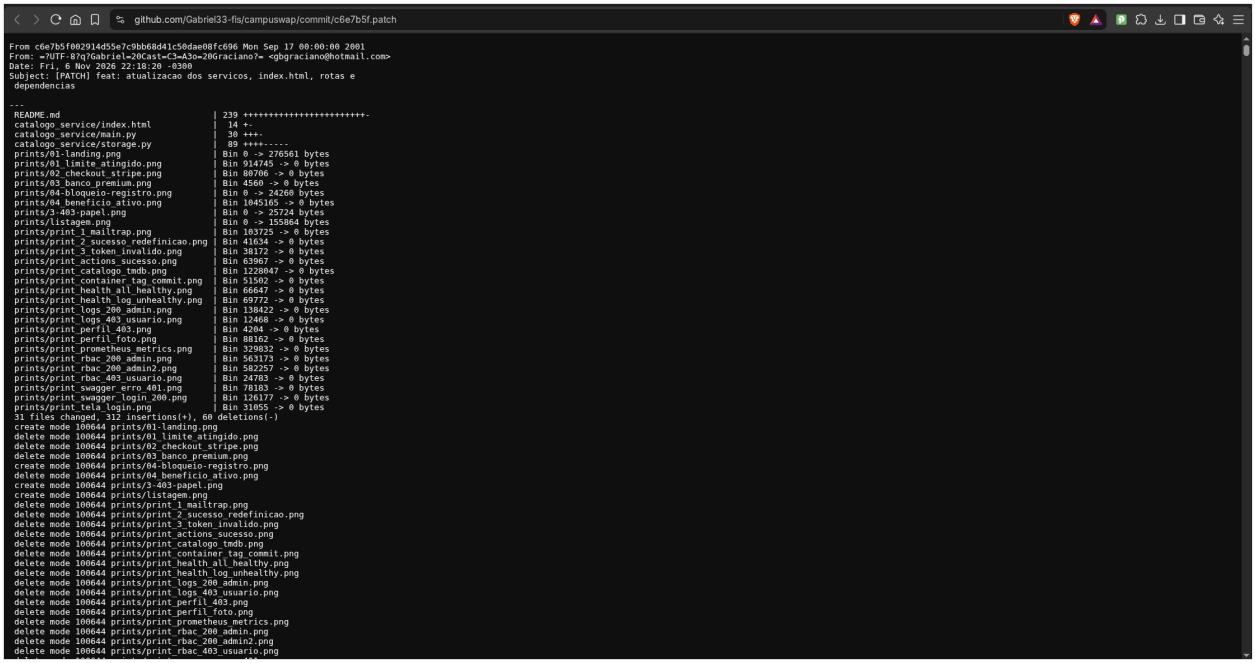


Figura 25: Atividade 10: `.patch` do commit c6e7b5f, com `Date: Fri, 6 Nov 2026 22:18:20 -0300`



Figura 26: Atividade 10: landing page da aplicação

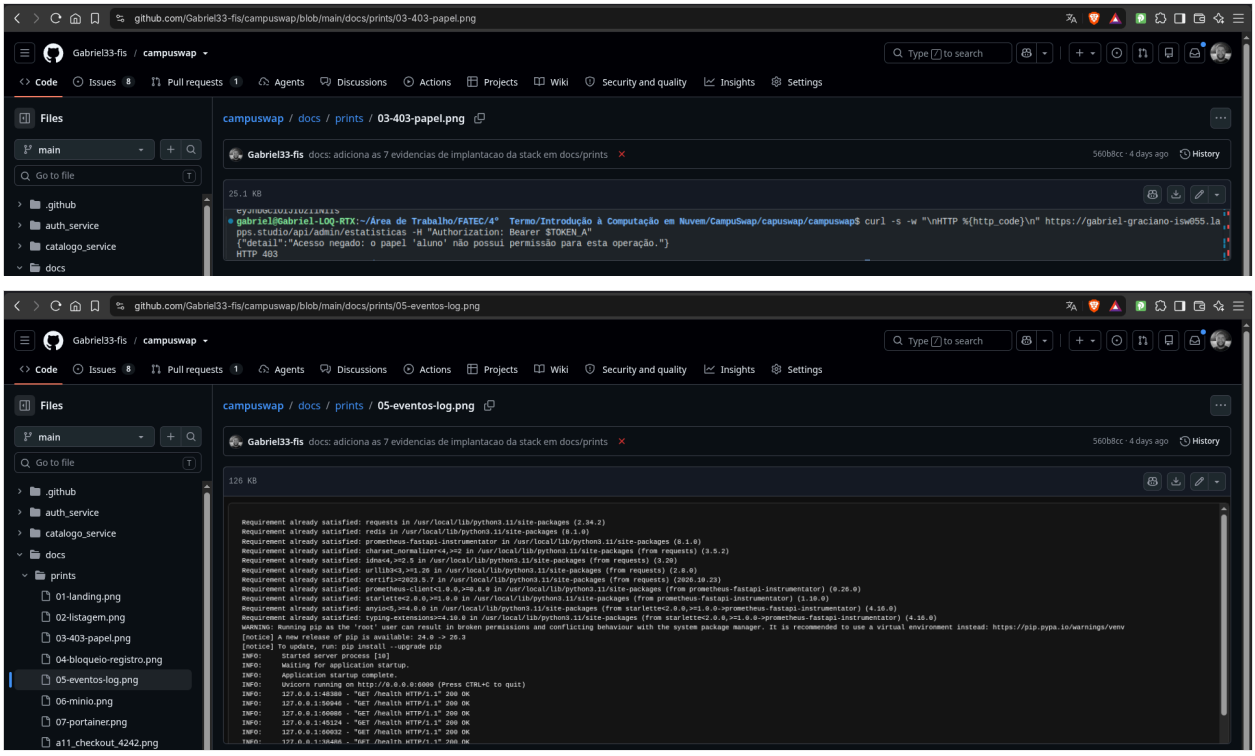


Figura 27: Atividade 10: respostas 403 por papel e eventos de auditoria (prints versionados no repositório do produto)

Dificuldades. Adaptar o `log_service` para eventos do novo domínio, sem quebrar o fluxo assíncrono já existente.

5.5 Atividade 11 — SaaS completo: planos, limites e assinatura com Stripe

Descrição	Plano Free com limite, plano Pro recorrente, checkout, webhooks atualizando o plano, portal do cliente e painel admin.
Data planejada	13/11/2026

Data realizada	11/11/2026 00:44
Situação	entregue no prazo
Evidência	Issue \#9 (11/11/2026 00:44), merge 3f4d2b9 (00:41) e prints do fluxo completo no ar
Link	https://github.com/Gabriel33-fis/campuswap/issues/9

O que foi feito. Implementei o plano Free (3 anúncios por mês, com resposta HTTP 402 ao exceder) e o plano Pro (R$ 9,90 por mês, sem limite), com Checkout Session em modo `subscription`, portal do cliente e webhooks que atualizam `plano`, `status_assinatura` e `is_premium`. O webhook usa o proxy do catálogo e a idempotência em `eventos_stripe`. Há também painel admin com visão geral e eventos recentes.

Como foi entregue. A issue #9 e o merge 3f4d2b9 foram feitos na madrugada de 11/11, com o fluxo validado no ambiente local. Em 11/11, durante o dia, recoloquei o site em produção (Portainer) e repeti o fluxo completo no domínio público; os prints abaixo de produção são dessa repetição e têm horário próprio, posterior à issue.

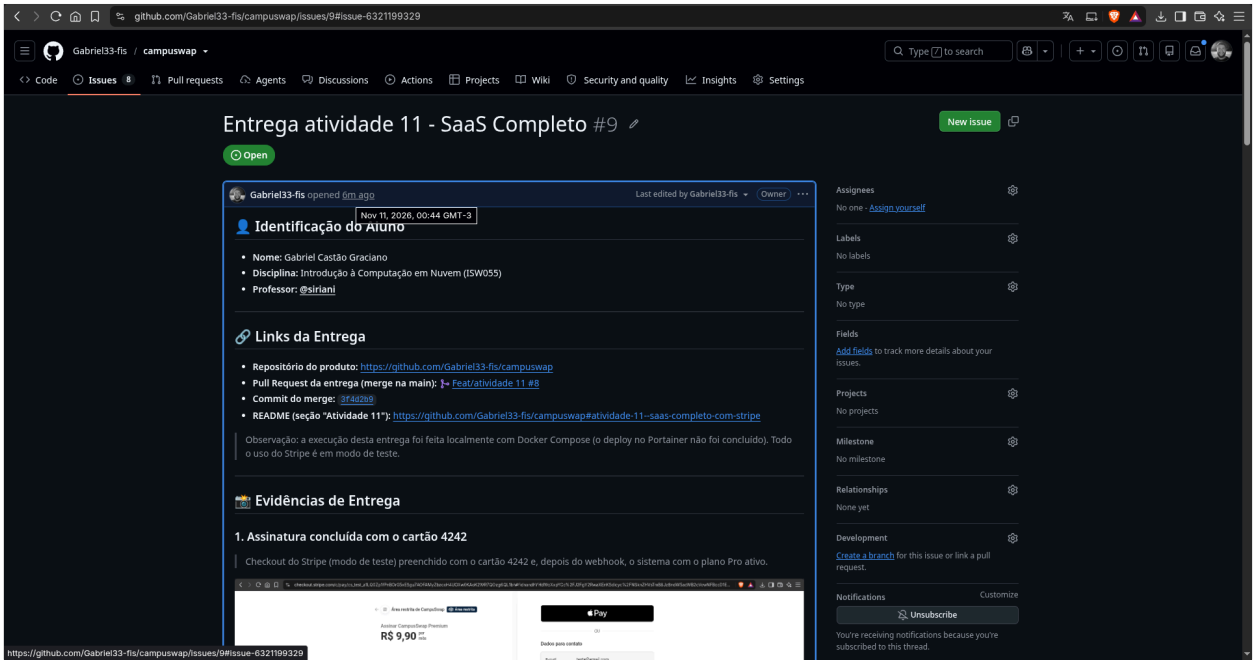


Figura 28: Atividade 11: issue #9 de entrega, com data e hora ao passar o mouse (11/11/2026 00:44 GMT-3)

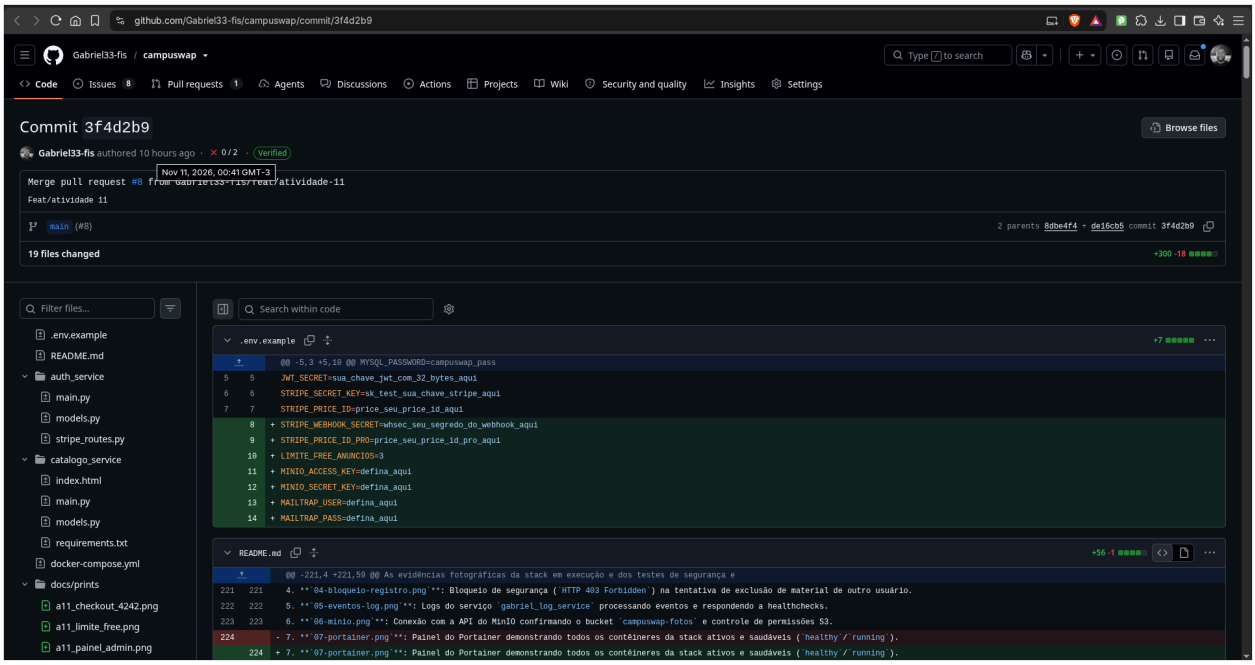


Figura 29: Atividade 11: commit 3f4d2b9 (merge da PR), com data e hora (11/11/2026 00:41 GMT-3)

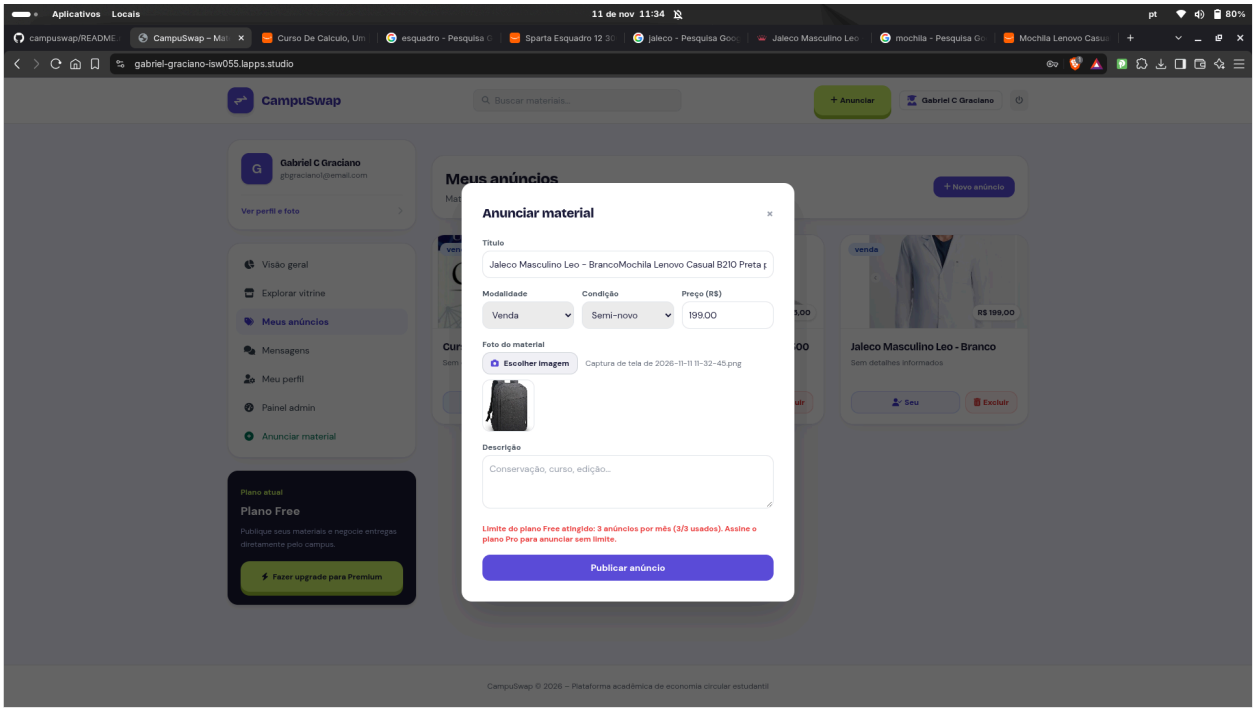


Figura 30: Atividade 11, produção, 11/11/2026 11:34: aluno Free ao tentar o 4º anúncio do mês, com o bloqueio “Limite do plano Free atingido: 3 anúncios por mês”

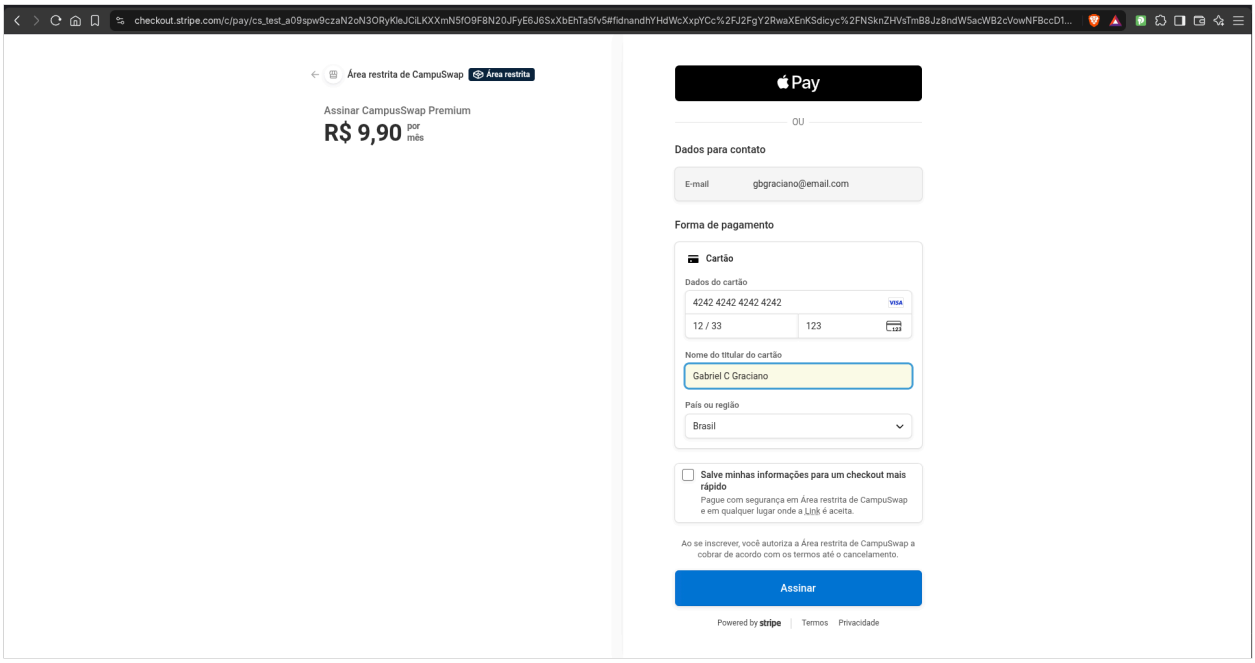
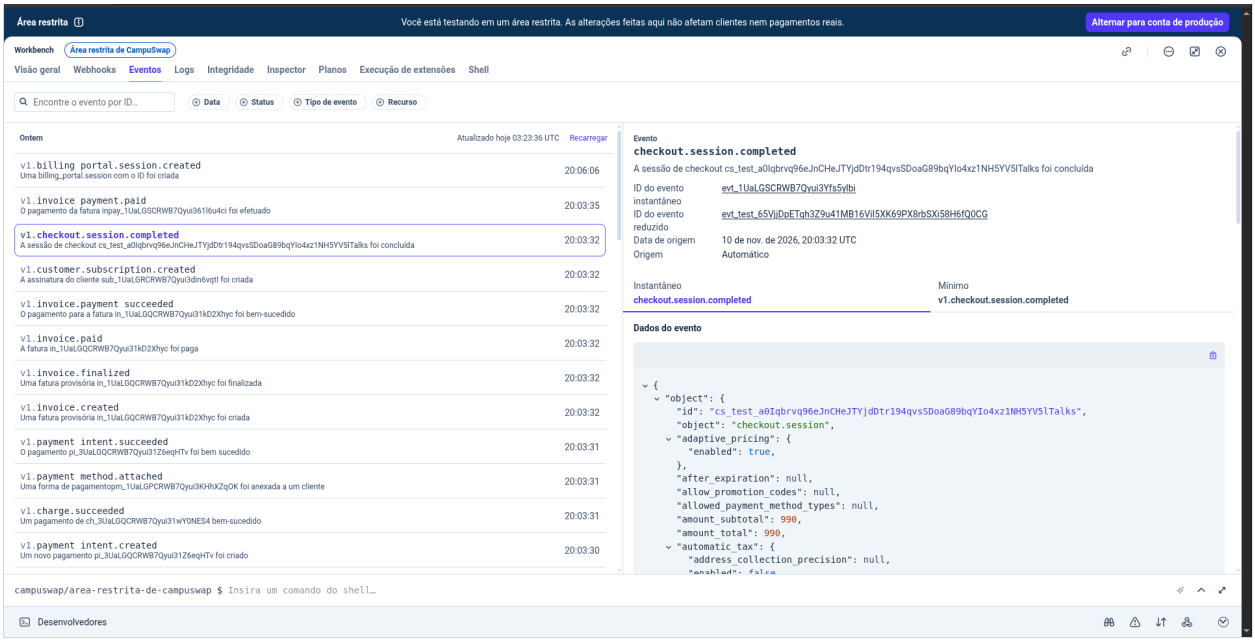


Figura 31: Atividade 11, produção: checkout de assinatura com o cartão de teste 4242 (R$ 9,90)



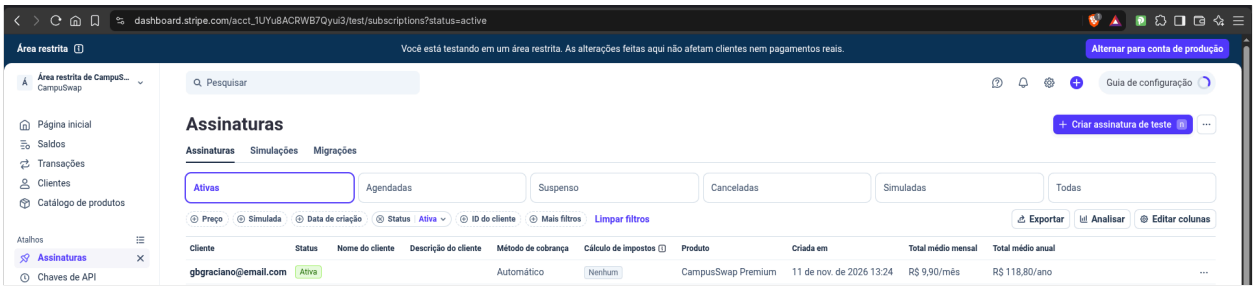


Figura 32: Atividade 11, Stripe: evento `checkout.session.completed` e a assinatura ativa

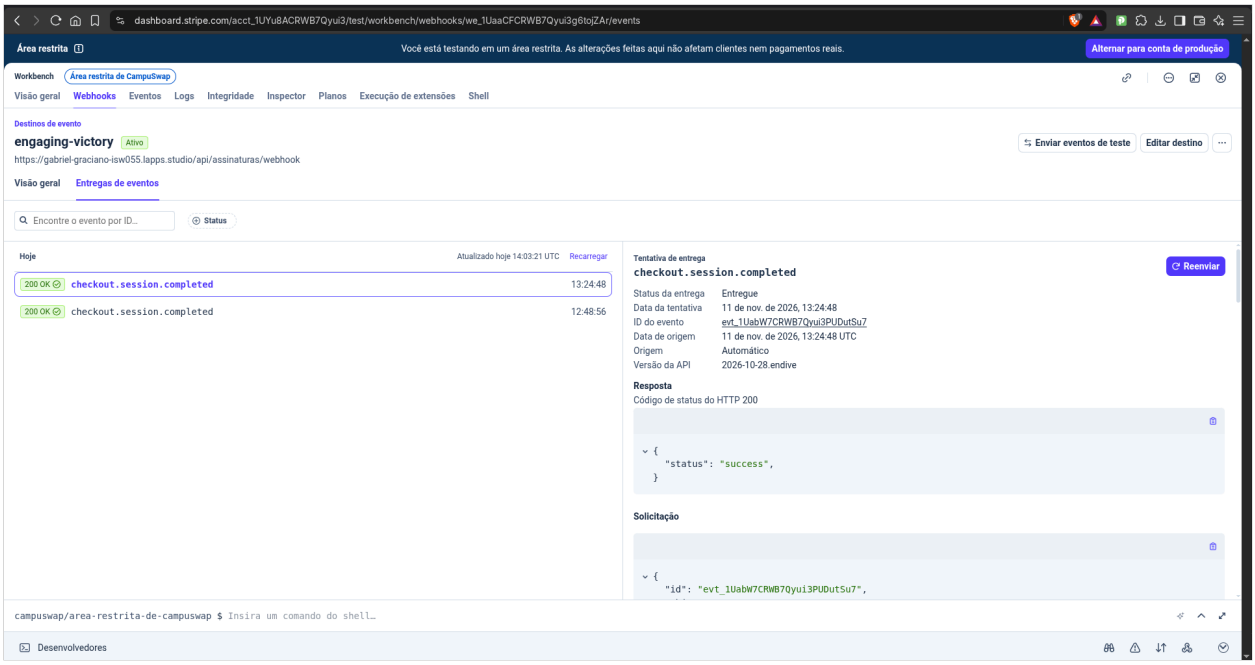


Figura 33: Atividade 11, Stripe: entregas do webhook ao endpoint do site, resposta 200

```
gabriel@Gabriel-LOQ-RTX:~/campuswap$ docker exec -it gabriel_mariadb mariadb -ucampuswap_user -p campuswap_db -e "SELECT event_id, tipo, p
rocessado_em FROM eventos_stripe ORDER BY id DESC LIMIT 10;"
Enter password:
+----------------------------+------------------------------+---------------------+
| event_id                   | tipo                         | processado_em       |
+----------------------------+------------------------------+---------------------+
| evt_1UaLIwCRWB7Qyui3JMdCrod5 | billing_portal.session.created | 2026-11-10 20:06:06 |
| evt_1UaLGVCRWB7Qyui3vOZL5mLz | invoice_payment.paid           | 2026-11-10 20:03:35 |
| evt_1UaLGTCRWB7Qyui3tl8LyXte | invoice.payment_succeeded      | 2026-11-10 20:03:33 |
| evt_3UaLGQCRWB7Qyui31Ar6QmOS | payment_intent.created         | 2026-11-10 20:03:33 |
| evt_1UaLGTCRWB7Qyui3c68RZUoa | invoice.created                | 2026-11-10 20:03:33 |
| evt_3UaLGQCRWB7Qyui31o5yRDU3 | payment_intent.succeeded       | 2026-11-10 20:03:33 |
| evt_1UaLGSCRWB7Qyui3BvxZIqfa | customer.subscription.created  | 2026-11-10 20:03:33 |
| evt_1UaLGSCRWB7Qyui3in1Bf3ZA | payment_method.attached        | 2026-11-10 20:03:33 |
| evt_1UaLGSCRWB7Qyui3UlTDJ8VP | invoice.paid                   | 2026-11-10 20:03:33 |
| evt_1UaLGSCRWB7Qyui3Yfs5ylbi | checkout.session.completed     | 2026-11-10 20:03:33 |
+----------------------------+------------------------------+---------------------+
```

Figura 34: Atividade 11, banco de dados: eventos processados em `eventos_stripe` (consulta feita no banco do ambiente local de desenvolvimento; horários em UTC)

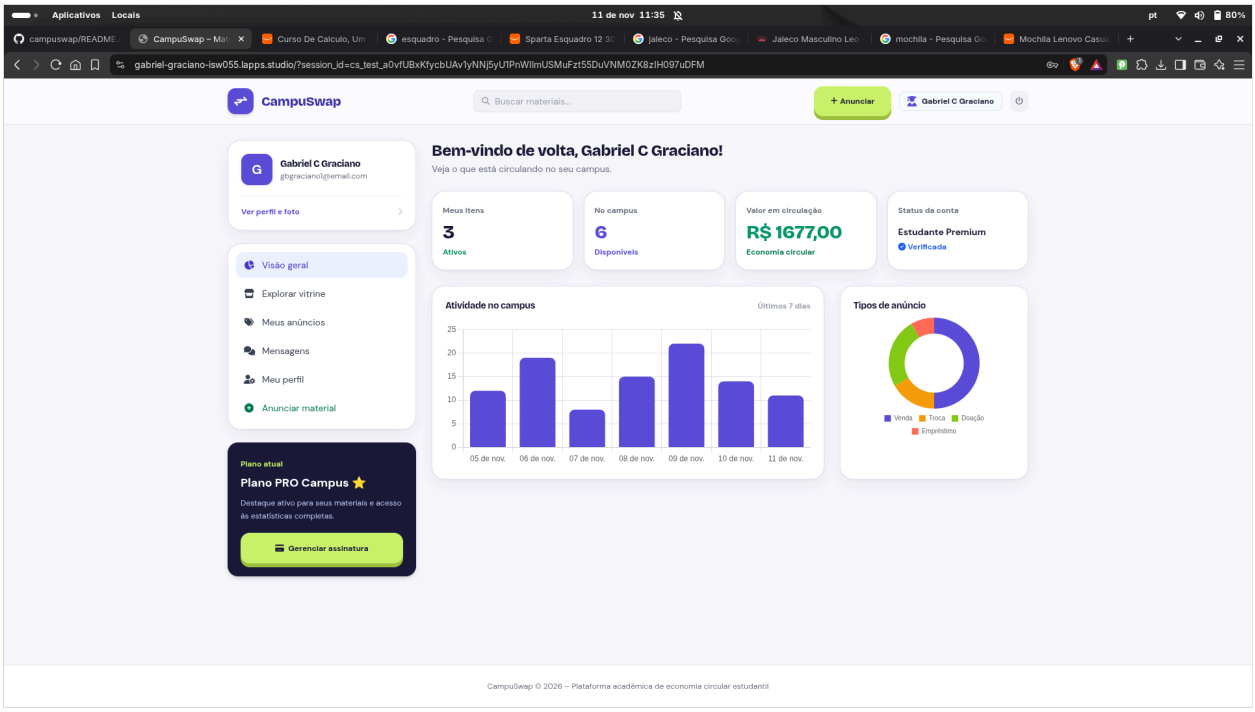


Figura 35: Atividade 11, produção, 11/11/2026 11:35: plano Pro ativo no perfil do aluno após o checkout

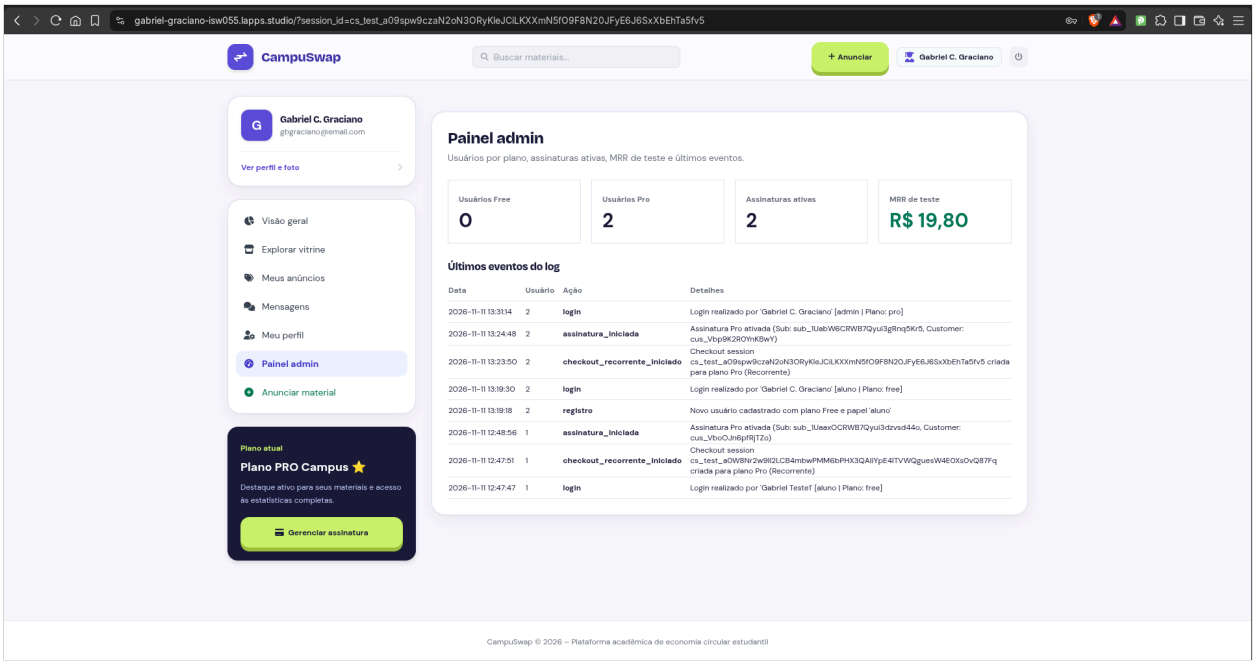


Figura 36: Atividade 11, produção: painel admin com o MRR e eventos de auditoria

Dificuldades. (1) O CI herdado do Tom Hanks falhava e impedia a publicação das imagens. A execução do commit de entrega 3f4d2b9 (11/11, 00:41) terminou com **falha** por causa disso; depois substituí o CI por compilação e publicação no GHCR, e as execuções posteriores, como a do commit 59ff0a9, passaram. (2) Ao recriar a stack, o banco recusava a senha nova porque o volume antigo do MariaDB mantinha a senha anterior; foi necessário recriar apenas o volume desta stack. Senhas com caracteres especiais também quebravam a URL de conexão. (3) O checkout falhava com `No such price` porque eu havia configurado o id do produto (`prod_...`) em vez do id do preço (`price_...`). (4) O webhook não chegava ao `auth_service`, que não é público; resolvi criando o proxy `POST /api/assinaturas/webhook` no catálogo (PR #11). (5) O cadastro aceitava `papel` enviado pelo cliente; corrigi para sempre criar `aluno` (PR #13).

6 Considerações finais

O trabalho mostrou, na prática, como separar responsabilidades em microsserviços e como cada decisão de infraestrutura (rede interna, variáveis de ambiente, volumes, limites de memória) aparece depois como um erro concreto de execução. A maior parte do tempo gasto no bimestre foi de diagnóstico: ler logs de contêiner, conferir códigos HTTP e comparar o que o ambiente local fazia com o que a produção fazia.

Duas lições ficaram claras. Primeiro, o ambiente de produção precisa ser reproduzível a partir do repositório: foi o `docker-compose.prod.yml` com CI/CD no GHCR que permitiu recolocar o site no ar em poucas horas. Segundo, a prova de que algo funciona é o teste de ponta a ponta (cadastro, limite, pagamento, webhook, plano), e não o fato de o código existir.

Como próximos passos: trocar o MinIO em memória (`S3BACKEND=mem`, que perde as fotos ao reiniciar) por armazenamento persistente, adicionar certificado TLS automático, incluir testes automatizados reais no CI, rotacionar as credenciais de teste que foram expostas durante o desenvolvimento e usar WebSockets no chat.

7 Declaração de autoria

Declaro que este relatório foi elaborado por mim, individualmente, e que as evidências apresentadas correspondem a entregas de minha autoria, verificáveis nos links informados. As entregas feitas com atraso estão identificadas como tal.

Gabriel Castão Graciano

Pompeia, 13/11/2026